

Template

Billy Wang

2026 年 9 月 3 日

目录

1	写在前面	3
1.1	基础模版	3
1.2	vimrc	3
2	数据结构	4
2.1	zkw 线段树	4
2.2	珂朵莉树	5
2.3	FHQ-Treap	5
2.4	并查集	6
2.5	ST 表	7
2.6	树状数组	8
2.7	线段树	8
2.7.1	我一般写的	8
2.7.2	一款较通用的	10
3	数学	12
3.1	快速幂	12
3.2	高斯消元	12
3.3	筛法	13
3.3.1	埃式筛	13
3.3.2	线性筛	13
3.4	类欧几里得	14
3.4.1	经典问题/我自己写的	14
3.4.2	洛谷模版题/OI-Wiki	15
3.5	递推组合数	16
3.6	矩阵快速幂	16
3.7	扩展欧几里得	17
3.8	快速莫比乌斯/沃尔什变换	18
3.9	子集卷积	20

4	图论	21
4.1	倍增	21
4.2	网络流	22
4.2.1	最大流	22
4.2.2	费用流	24
4.3	二分图最大匹配	26
4.4	Tarjan 强连通分量缩点	27
4.5	树直径	27
4.6	树重心	28
4.7	树链剖分	29
4.8	最短路	31
4.8.1	Floyd (最小环)	31
4.8.2	Spfa (判负环)	31
4.8.3	Dijkstra	32
4.9	拓扑排序	33
4.10	最小生成树	33
4.11	欧拉路径/回路	34
5	字符串	34
5.1	KMP	34
5.2	Trie 树	35
5.3	Manacher	35
5.4	回文自动机	37
5.5	字符串哈希	38
5.6	后缀数组	40
5.7	AC 自动机	41
6	计算几何	43
6.1	计算几何模板 - 吉如一	43
7	STL	60
7.1	算法库	60

1 写在前面

1.1 基础模版

```
1 #include <bits/stdc++.h>
2 using namespace std;
3 typedef long long ll;
4 #define REP( i, a, b ) for ( int i = ( a ); i <= ( b ); ++i )
5 #define REPD( i, a, b ) for ( int i = ( a ); i >= ( b ); --i )
6 inline ll rd() { /* {{{ */
7     ll r = 0, k = 1;
8     char c;
9     while ( !isdigit( c = getchar() ) )
10         if ( c == '-' )
11             k = -k;
12     while ( isdigit( c ) )
13         r = r * 10 + c - '0', c = getchar();
14     return r * k;
15 }
16 inline void prt( ll x ) {
17     printf( "%lld ", x );
18 } /* }}} */
19
20 int main() {
21
22     return 0;
23 }
```

1.2 vimrc

```
1 syntax on
2 set ts=2 sw=2 sts=2
3 set et ai cin nu sm ru cul
4 set mouse=a title wim=list nowrap ls=2
5 set ml sc
6 set mls=5
7 set fdm=marker
8 colo desert
9
10 im <C-S> <esc>:w<CR>
```

```

11 nn <C-S> :w<CR>
12 nn <space> za
13
14 " 使用 Windows 环境时可能需要
15 "     set shell=powershell
16 "     set backspace=indent,eol,start
17 "     set nocompatible
18 " set sh=powershell bs=indent,eol,start nosp
19 "
20 " 如果 <C-S> 导致 vim 卡死
21 " 在 .bashrc 里加一句 stty -ixon

```

2 数据结构

2.1 zkw 线段树

单点修区间查

```

1 ll s[N << 2], a[N];
2 int M;
3
4 ll f( ll x, ll y ) {
5     return x + y; // 改这
6 }
7
8 void build() {
9     for ( M = 1; M <= n + 1; M <= 1 ) {}
10    REP( i, 1, n ) s[i + M] = a[i];
11    REPD( i, M - 1, 1 ) s[i] = f( s[2 * i], s[2 * i + 1] );
12 }
13
14 ll qrang( int l, int r, ll init ) { // 根据 f 传 init
15     ll res = init;
16     for ( l = l + M - 1, r = r + M + 1; l ^ r ^ 1; l >>= 1, r >>= 1 )
17     {
18         if ( ~l & 1 )
19             res = f( res, s[l ^ 1] );
20         if ( r & 1 )
21             res = f( res, s[r ^ 1] );
22     }
23     return res;

```

```

23 }
24
25 void edit( int x, ll v ) {
26     for ( s[x += M] = v, x >>= 1; x; x >>= 1 ) {
27         s[x] = f( s[2 * x], s[2 * x + 1] );
28     }
29 }
30
31 ll qpoint( int x ) {
32     return s[x + M];
33 }

```

2.2 珂朵莉树

```

1 struct node {
2     int l, r;
3     mutable int v;
4     bool operator<( const node& rhs ) const { return l < rhs.l; }
5 };
6
7 set<node> odt;
8 typedef set<node>::iterator iter;
9
10 iter split( ll p ) {
11     iter tmp = odt.lower_bound( (node) { p, 0, 0 } );
12     if ( tmp != odt.end() && tmp->l == p )
13         return tmp;
14     --tmp;
15     int tl = tmp->l, tr = tmp->r, tv = tmp->v;
16     odt.erase( tmp );
17     odt.insert( (node) { tl, p - 1, tv } );
18     return odt.insert( (node) { p, tr, tv } ).first;
19 }
20
21 // 修改和查询注意 split 顺序
22 // iter itr=split(r+1), itl=split(l);

```

2.3 FHQ-Treap

以模版文艺平衡树为例

```

1  int n, m, clk, rt;
2  struct node {
3      int key, val, sz, tag, ls, rs;
4  } t[N];
5  int newnode( int k ) {
6      return t[++clk] = (node) { k, rand(), 1, 0 }, clk;
7  }
8  void down( int o ) {
9      if ( t[o].tag ) {
10         t[t[o].ls].tag = 1 - t[t[o].ls].tag;
11         t[t[o].rs].tag = 1 - t[t[o].rs].tag;
12         swap( t[t[o].ls].ls, t[t[o].ls].rs );
13         swap( t[t[o].rs].ls, t[t[o].rs].rs );
14         t[o].tag = 0;
15     }
16 }
17 void up( int o ) {
18     t[o].sz = t[t[o].ls].sz + t[t[o].rs].sz + 1;
19 }
20 void split( int o, int x, int& L, int& R ) {
21     if ( o == 0 )
22         return L = R = 0, void();
23     down( o );
24     if ( t[t[o].ls].sz + 1 >= x )
25         R = o, split( t[o].ls, x, L, t[o].ls );
26     else
27         L = o, split( t[o].rs, x - t[t[o].ls].sz - 1, t[o].rs, R );
28     up( o );
29 }
30 int merge( int L, int R ) {
31     if ( L == 0 || R == 0 )
32         return L + R;
33     if ( t[L].val > t[R].val )
34         return down( L ), t[L].rs = merge( t[L].rs, R ), up( L ), L;
35     else
36         return down( R ), t[R].ls = merge( L, t[R].ls ), up( R ), R;
37 }

```

2.4 并查集

```

1 ll n, fa[N];
2 void init() {
3     iota( fa + 1, fa + n + 1, 1 );
4 }
5
6 int find( int x ) {
7     if ( x == fa[x] )
8         return x;
9     return fa[x] = find( fa[x] );
10 }
11
12 void merge( int x, int y ) {
13     x = find( x ), y = find( y );
14     if ( x != y )
15         fa[x] = y;
16 }

```

2.5 ST 表

```

1 ll n, a[N], st[N][SP + 10], to[N][SP + 10], l2g[N];
2 ll op( ll x, ll y ) {
3     return max( x, y );
4 }
5
6 void init() {
7     l2g[1] = 0, to[n + 1][0] = n + 1;
8     REP( i, 2, n ) l2g[i] = l2g[i - 1] + !( i & ( i - 1 ) );
9     REP( i, 1, n ) st[i][0] = a[i], to[i][0] = i + 1;
10    REP( i, 1, SP ) {
11        REP( j, 1, n ) {
12            to[j][i] = to[to[j][i - 1]][i - 1];
13            st[j][i] = op( st[j][i - 1], st[to[j][i - 1]][i - 1] );
14        }
15    }
16 }
17
18 ll query( ll l, ll r ) { // [l, r] 闭区间
19     ++r;
20     ll d = l2g[r - l];
21     return op( st[l][d], st[r - ( 1 << d )][d] );

```

```
22 }
```

2.6 树状数组

```
1 ll n, fwt[N];
2 ll prod( ll x, ll d ) {
3     return x + d;
4 }
5 ll op( ll x, ll y ) {
6     return x + y;
7 }
8
9 void edit( int x, ll d ) {
10     for ( ; x <= n; x += x & -x )
11         fwt[x] = prod( fwt[x], d );
12 }
13
14 ll query( int x ) {
15     assert( 1 <= x && x <= n );
16     ll res = fwt[x];
17     x -= x & -x;
18     // 这种写法不用考虑最大或最小值的初值问题
19     for ( ; x; x -= x & -x )
20         res = op( res, fwt[x] );
21     return res;
22 }
```

2.7 线段树

2.7.1 我一般写的

```
1 ll n, a[N];
2 struct seg {
3     int l, r;
4     ll sum, tag;
5 } s[N << 2];
6 #define lson ( o * 2 )
7 #define rson ( o * 2 + 1 )
8
9 void down( int o ) {
```

```

10     if ( s[o].tag ) {
11         s[lson].sum += s[o].tag * ( s[lson].r - s[lson].l + 1 ),
12         s[lson].tag += s[o].tag;
13         s[rson].sum += s[o].tag * ( s[rson].r - s[rson].l + 1 ),
14         s[rson].tag += s[o].tag;
15         s[o].tag = 0;
16     }
17 }
18
19 void build( int o, int l, int r ) {
20     s[o].l = l, s[o].r = r;
21     if ( l == r )
22         return s[o].sum = a[l], void();
23     int mid = ( l + r ) / 2;
24     build( lson, l, mid );
25     build( rson, mid + 1, r );
26     s[o].sum = s[lson].sum + s[rson].sum;
27 }
28
29 void add( int o, int l, int r, ll d ) {
30     if ( l <= s[o].l && s[o].r <= r )
31         return s[o].sum += d * ( s[o].r - s[o].l + 1 ), s[o].tag += d,
32         void();
33     down( o );
34     int mid = ( s[o].l + s[o].r ) / 2;
35     if ( l <= mid )
36         add( lson, l, r, d );
37     if ( r > mid )
38         add( rson, l, r, d );
39     s[o].sum = s[lson].sum + s[rson].sum;
40 }
41
42 ll sum( int o, int l, int r ) {
43     if ( l <= s[o].l && s[o].r <= r )
44         return s[o].sum;
45     down( o );
46     int mid = ( s[o].l + s[o].r ) / 2;
47     ll res = 0;
48     if ( l <= mid )
49         res += sum( lson, l, r );

```

```

49     if ( r > mid )
50         res += sum( rson, l, r );
51     return res;
52 }

```

2.7.2 一款较通用的

```

1  ll n, a[N];
2
3  namespace sg {
4
5  struct Q { // 存修改, 存 tag
6      ll tag;
7      Q( ll tag = 0 ) : tag( tag ) {} // 注意初始值
8      void operator+=( const Q& q ) { tag += q.tag; }
9      // tag 的叠加
10 };
11 struct P { // 存记录维护的答案
12     ll sum;
13     P( ll sum = 0 ) : sum( sum ) {} // 注意初始值
14     void push( const Q& q, int l, int r ) { sum += q.tag * ( r - l +
15         1 ); }
16     // 区间从操作更新答案
17     void init( int x ) { sum = a[x]; } // 建树中的赋值
18 };
19 P operator&( const P& a, const P& b ) { // 子区间合并答案
20     return P( a.sum + b.sum );
21 }
22 // 基本上只需要修改上面的部分
23 P p[N << 2];
24 Q q[N << 2];
25 #define lson o * 2, l, ( l + r ) / 2
26 #define rson o * 2 + 1, ( l + r ) / 2 + 1, r
27 void up( int o, int l, int r ) {
28     if ( l < r )
29         p[o] = p[o * 2] & p[o * 2 + 1];
30 }
31 void push( const Q& v, int o, int l, int r ) {
32     q[o] += v;
33     p[o].push( v, l, r );

```

```

33 }
34 void down( int o, int l, int r ) {
35     push( q[o], lson );
36     push( q[o], rson );
37     q[o] = Q();
38 }
39 void build( int o = 1, int l = 1, int r = n ) {
40     if ( l == r )
41         p[o].init( l );
42     else {
43         build( lson );
44         build( rson );
45     }
46     up( o, l, r );
47 }
48 P query( int ql, int qr, int o = 1, int l = 1, int r = n ) {
49     if ( ql > r || l > qr )
50         return P();
51     if ( ql <= l && r <= qr )
52         return p[o];
53     down( o, l, r );
54     return query( ql, qr, lson ) & query( ql, qr, rson );
55 }
56 void update( int ql, int qr, const Q& v, int o = 1, int l = 1, int
    r = n ) {
57     if ( ql > r || l > qr )
58         return;
59     if ( ql <= l && r <= qr ) {
60         push( v, o, l, r );
61         return;
62     }
63     down( o, l, r );
64     update( ql, qr, v, lson );
65     update( ql, qr, v, rson );
66     up( o, l, r );
67 }
68
69 }
70 using namespace sg;

```

3 数学

3.1 快速幂

```
1 const ll MOD = 998244353; // 改模数
2
3 ll qpow( ll a, ll x ) {
4     ll res = 1;
5     a %= MOD;
6     while ( x ) {
7         if ( x & 1 )
8             res = res * a % MOD;
9         a = a * a % MOD, x >>= 1;
10    }
11    return res;
12 }
13
14 ll inv( ll x ) {
15     return qpow( x, MOD - 2 );
16 } // 模数为质数时
```

3.2 高斯消元

```
1 const int N = 110;
2 ll n;
3 double a[N][N], b[N];
4 void work() {
5     n = rd();
6     REP( i, 1, n ) {
7         REP( j, 1, n ) a[i][j] = rd();
8         b[i] = rd();
9     }
10    REP( i, 1, n ) {
11        int t = i;
12        REP( j, i + 1, n )
13            if ( abs( a[j][i] ) > 1e-7
14                && ( abs( a[t][i] ) > abs( a[j][i] ) || abs( a[t][i] ) < 1
15                    e-7 ) )
16            t = j;
17        REP( j, i, n ) swap( a[t][j], a[i][j] );
18    }
```

```

17     if ( abs( a[i][i] ) < 1e-7 ) {
18         puts( "No Solution" );
19         return 0;
20     }
21     swap( b[t], b[i] );
22     double e = a[i][i];
23     REP( j, i, n ) a[i][j] /= e;
24     b[i] /= e;
25     REP( j, i + 1, n ) {
26         double d = a[j][i];
27         REP( k, i, n ) a[j][k] -= d * a[i][k];
28         b[j] -= d * b[i];
29     }
30 }
31 REPd( i, n, 1 ) REP( j, 1, i - 1 ) b[j] -= a[j][i] * b[i], a[j][i]
    ] = 0;
32 // REP( i, 1, n ) printf( "%.2f\n", b[i] );
33 // b[1...n] 保存 Ax=b 的解
34 }

```

3.3 筛法

3.3.1 埃式筛

```

1  bitset<N> b;
2  ll n;
3  vector<ll> prime;
4  void erato() {
5      REP( i, 2, n ) if ( !b[i] ) {
6          prime.push_back( i );
7          for ( int j = i + i; j <= n; j += i )
8              b[j] = 1;
9      }
10 }

```

3.3.2 线性筛

```

1  bitset<N> b;
2  ll n, phi[N];
3  vector<ll> prime;
4  void euler() {

```

```

5  REP( i, 2, n ) {
6      if ( !b[i] ) {
7          prime.push_back( i );
8          phi[i] = i - 1;
9      }
10     for ( int p : prime ) {
11         if ( p * i > n )
12             break;
13         b[p * i] = 1;
14         if ( i % p == 0 ) {
15             phi[p * i] = phi[i] * p;
16             break;
17         }
18         phi[p * i] = phi[i] * phi[p];
19     }
20 }
21 }

```

3.4 类欧几里得

3.4.1 经典问题/我自己写的

```

1  ll floordiv( ll x, ll y ) {
2      assert( y != 0 );
3      if ( y < 0 )
4          x = -x;
5      return x >= 0 ? ( x / y ) : ( ( x - y + 1 ) / y );
6  }
7
8  ll mod( ll x, ll y ) {
9      return ( x % y + y ) % y;
10 }
11
12 ll f( ll a, ll b, ll c, ll n ) {
13     if ( c < 0 )
14         a = -a, b = -b, c = -c;
15     assert( c != 0 );
16     ll res = ( floordiv( a, c ) % MOD + MOD ) % MOD
17             * ( ( n * ( n + 1 ) / 2 ) % MOD ) % MOD
18             + ( floordiv( b, c ) % MOD + MOD ) % MOD * ( n + 1 ) %
19               MOD;

```

```

19     res %= MOD;
20     a = mod( a, c ), b = mod( b, c );
21     if ( !a )
22         return res;
23     res += ( n + 1 ) * ( floordiv( a * n + b, c ) % MOD ) % MOD
24         - floordiv( b, a ) % MOD;
25     res = ( res % MOD + MOD ) % MOD;
26     res -= f( c, a - b - 1, a, floordiv( a * n + b, c ) );
27     res = ( res % MOD + MOD ) % MOD;
28     return res;
29 }

```

3.4.2 洛谷模版题/OI-Wiki

$$f(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor$$

$$g(a, b, c, n) = \sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor$$

$$h(a, b, c, n) = \sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2$$

```

1 struct Data {
2     int f, g, h;
3     // f(a, b, c, n)=\sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor
4     // g(a, b, c, n)=\sum_{i=0}^n i \lfloor \frac{ai+b}{c} \rfloor
5     // h(a, b, c, n)=\sum_{i=0}^n \lfloor \frac{ai+b}{c} \rfloor^2
6 };
7
8 Data solve( long long a, long long b, long long c, long long n ) {
9     constexpr long long M = 998244353;
10    constexpr long long i2 = ( M + 1 ) / 2;
11    constexpr long long i6 = ( M + 1 ) / 6;
12    long long n2 = ( n + 1 ) * n % M * i2 % M;
13    long long n3 = ( 2 * n + 1 ) * ( n + 1 ) % M * n % M * i6 % M;
14    Data res = { 0, 0, 0 };
15    if ( a >= c || b >= c ) {
16        auto tmp = solve( a % c, b % c, c, n );
17        long long aa = a / c, bb = b / c;
18        res.f = ( tmp.f + aa * n2 + bb * ( n + 1 ) ) % M;
19        res.g = ( tmp.g + aa * n3 + bb * n2 ) % M;
20        res.h
21            = ( tmp.h + 2 * bb * tmp.f % M + 2 * aa * tmp.g % M + aa * aa
22                % M * n3 % M
23                + bb * bb % M * ( n + 1 ) % M + 2 * aa * bb % M * n2 % M

```

```

        )
23         % M;
24     return res;
25 }
26 long long m = ( a * n + b ) / c;
27 if ( !m )
28     return res;
29 auto tmp = solve( c, c - b - 1, a, m - 1 );
30 res.f = ( m * n - tmp.f + M ) % M;
31 res.g = ( m * n2 + ( M - tmp.f ) * i2 + ( M - tmp.h ) * i2 ) % M;
32 res.h = ( n * m % M * m - tmp.f - tmp.g * 2 + 3 * M ) % M;
33 return res;
34 }

```

3.5 递推组合数

```

1 ll C[N][N];
2 void calcC( int n ) {
3     C[0][0] = 1;
4     REP( i, 1, n ) {
5         C[i][0] = C[i][i] = 1;
6         REP( j, 1, i - 1 ) C[i][j] = ( C[i - 1][j] + C[i - 1][j - 1] )
            % MOD;
7     }
8 }

```

3.6 矩阵快速幂

```

1 struct mat {
2     ll n, m;
3     vector<vector<ll>> val; // 注意下标从 0 开始
4     mat( ll _n, ll _m ) : n( _n ), m( _m ) {
5         val.resize( _n );
6         REP( i, 0, _n - 1 ) val[i].resize( _m, 0 );
7     }
8     mat( ll _n, ll _m, vector<vector<ll>> _val )
9         : n( _n ), m( _m ), val( _val ) {}
10 };
11
12 mat mul( const mat& x, const mat& y, ll mod ) {

```

```

13  assert( x.m == y.n );
14  mat res( x.n, y.m );
15  REP( i, 0, res.n - 1 ) REP( j, 0, res.m - 1 ) {
16      REP( k, 0, x.m - 1 ) {
17          res.val[i][j] = ( res.val[i][j] + x.val[i][k] * y.val[k][j] %
18                          mod ) % mod;
19      }
20  }
21  return res;
22 }
23 mat qpow( mat a, ll x, ll mod ) {
24     assert( a.n == a.m );
25     mat res( a.n, a.n );
26     REP( i, 0, a.n - 1 ) res.val[i][i] = 1;
27     REP( i, 0, a.n - 1 ) REP( j, 0, a.m - 1 ) a.val[i][j] %= mod;
28     while ( x ) {
29         if ( x & 1 )
30             res = mul( res, a, mod );
31         a = mul( a, a, mod ), x >>= 1;
32     }
33     return res;
34 }

```

3.7 扩展欧几里得

```

1  ll exgcd( ll a, ll b, ll& x, ll& y ) {
2      if ( b == 0 ) {
3          x = 1, y = 0;
4          return a;
5      }
6      ll xx = 0, yy = 0;
7      ll res = exgcd( b, a % b, xx, yy );
8      y = xx - ( a / b ) * yy, x = yy;
9      return res;
10 }
11
12 ll inv( ll x, ll mod ) {
13     ll xx, yy;
14     ll d = exgcd( x, mod, xx, yy );

```

```

15     assert( d == 1 );
16     return ( xx % mod + mod ) % mod;
17 }

```

3.8 快速莫比乌斯/沃尔什变换

```

1  int n, m;
2  ll a[N], b[N], c[N];
3  ll inv2;
4
5  void solor() {
6      for ( int i = 0; i < n; ++i ) {
7          for ( int j = 0; j < m; ++j ) {
8              if ( ( j >> i ) & 1 )
9                  continue;
10             ( a[j + ( 1 << i )] += a[j] ) %= MOD;
11             ( b[j + ( 1 << i )] += b[j] ) %= MOD;
12         }
13     }
14     for ( int j = 0; j < m; ++j )
15         c[j] = ( a[j] * b[j] ) % MOD;
16     for ( int i = 0; i < n; ++i ) {
17         for ( int j = 0; j < m; ++j ) {
18             if ( ( j >> i ) & 1 )
19                 continue;
20             a[j + ( 1 << i )] = ( a[j + ( 1 << i )] - a[j] + MOD ) % MOD;
21             b[j + ( 1 << i )] = ( b[j + ( 1 << i )] - b[j] + MOD ) % MOD;
22             c[j + ( 1 << i )] = ( c[j + ( 1 << i )] - c[j] + MOD ) % MOD;
23         }
24     }
25     for ( int j = 0; j < m; ++j )
26         prt( c[j] );
27     puts( "" );
28 }
29
30 void soland() {
31     for ( int i = 0; i < n; ++i ) {
32         for ( int j = 0; j < m; ++j ) {
33             if ( ( j >> i ) & 1 )
34                 continue;

```

```

35     ( a[j] += a[j + ( 1 << i )] ) %= MOD;
36     ( b[j] += b[j + ( 1 << i )] ) %= MOD;
37 }
38 }
39 for ( int j = 0; j < m; ++j )
40     c[j] = ( a[j] * b[j] ) % MOD;
41 for ( int i = 0; i < n; ++i ) {
42     for ( int j = 0; j < m; ++j ) {
43         if ( ( j >> i ) & 1 )
44             continue;
45         a[j] = ( a[j] - a[j + ( 1 << i )] + MOD ) % MOD;
46         b[j] = ( b[j] - b[j + ( 1 << i )] + MOD ) % MOD;
47         c[j] = ( c[j] - c[j + ( 1 << i )] + MOD ) % MOD;
48     }
49 }
50 for ( int j = 0; j < m; ++j )
51     prt( c[j] );
52 puts( "" );
53 }
54
55 void solxor() {
56     for ( int i = 0; i < n; ++i ) {
57         for ( int j = 0; j < m; ++j ) {
58             if ( ( j >> i ) & 1 )
59                 continue;
60             ( a[j] += a[j + ( 1 << i )] ) %= MOD;
61             a[j + ( 1 << i )] = ( a[j] - 2 * a[j + ( 1 << i )] + 2 * MOD
62                 ) % MOD;
63             ( b[j] += b[j + ( 1 << i )] ) %= MOD;
64             b[j + ( 1 << i )] = ( b[j] - 2 * b[j + ( 1 << i )] + 2 * MOD
65                 ) % MOD;
66         }
67     }
68     for ( int j = 0; j < m; ++j )
69         c[j] = ( a[j] * b[j] ) % MOD;
70     for ( int i = 0; i < n; ++i ) {
71         for ( int j = 0; j < m; ++j ) {
72             if ( ( j >> i ) & 1 )
73                 continue;
74             a[j] = ( ( a[j] + a[j + ( 1 << i )] ) * inv2 ) % MOD;

```

```

73     a[j + ( 1 << i )] = ( a[j] - a[j + ( 1 << i )] + MOD ) % MOD;
74     b[j] = ( ( b[j] + b[j + ( 1 << i )] ) * inv2 ) % MOD;
75     b[j + ( 1 << i )] = ( b[j] - b[j + ( 1 << i )] + MOD ) % MOD;
76     c[j] = ( ( c[j] + c[j + ( 1 << i )] ) * inv2 ) % MOD;
77     c[j + ( 1 << i )] = ( c[j] - c[j + ( 1 << i )] + MOD ) % MOD;
78 }
79 }
80 for ( int j = 0; j < m; ++j )
81     prt( c[j] );
82 puts( "" );
83 }

```

3.9 子集卷积

```

1  const ll MOD = 1e9 + 9;
2  const int N = 1050576, SP = 25;
3  int n, m;
4  int popcnt[N];
5  ll a[SP][N], b[SP][N], c[SP][N];
6  ll inv2;
7
8  void sol() {
9      for ( int k = 0; k <= n; ++k ) {
10         for ( int i = 0; i < n; ++i ) {
11             for ( int j = 0; j < m; ++j ) {
12                 if ( ( j >> i ) & 1 )
13                     continue;
14                 ( a[k][j + ( 1 << i )] += a[k][j] ) %= MOD;
15                 ( b[k][j + ( 1 << i )] += b[k][j] ) %= MOD;
16             }
17         }
18     }
19     for ( int k = 0; k <= n; ++k ) {
20         for ( int l = 0; l <= k; ++l ) {
21             for ( int j = 0; j < m; ++j ) {
22                 c[k][j] = ( c[k][j] + a[l][j] * b[k - l][j] % MOD ) % MOD;
23             }
24         }
25     }
26     for ( int k = 0; k <= n; ++k ) {

```

```

27     for ( int i = 0; i < n; ++i ) {
28         for ( int j = 0; j < m; ++j ) {
29             if ( ( j >> i ) & 1 )
30                 continue;
31             c[k][j + ( 1 << i )] = ( c[k][j + ( 1 << i )] - c[k][j] +
32                 MOD ) % MOD;
33         }
34     }
35     for ( int j = 0; j < m; ++j )
36         prt( c[popcnt[j]][j] );
37     puts( "" );
38 }

```

4 图论

4.1 倍增

```

1 void dfs( int x, int fa ) {
2     pa[x][0] = fa;
3     dep[x] = dep[fa] + 1;
4     REP( i, 1, SP ) pa[x][i] = pa[pa[x][i - 1]][i - 1];
5     for ( int& v : g[x] )
6         if ( v != fa ) {
7             dfs( v, x );
8         }
9 }
10
11 int lca( int x, int y ) {
12     if ( dep[x] < dep[y] )
13         swap( x, y );
14     int t = dep[x] - dep[y];
15     REP( i, 0, SP ) if ( t & ( 1 << i ) ) x = pa[x][i];
16     REPd( i, SP - 1, -1 ) {
17         int xx = pa[x][i], yy = pa[y][i];
18         if ( xx != yy )
19             x = xx, y = yy;
20     }
21     return x == y ? x : pa[x][0];
22 }

```

4.2 网络流

不是我写的，但是看着还好

其中 11 是我改的，不敢保证有没有漏改，但是过了洛谷模版题

4.2.1 最大流

```
1  constexpr ll INF = LLONG_MAX / 2;
2
3  struct E {
4      int to;
5      ll cp;
6      E( int to, ll cp ) : to( to ), cp( cp ) {}
7  };
8
9  struct Dinic {
10     static const int M = 1E5 * 5;
11     int m, s, t;
12     vector<E> edges;
13     vector<int> G[M];
14     int d[M];
15     int cur[M];
16
17     void init( int n, int s, int t ) {
18         this->s = s;
19         this->t = t;
20         for ( int i = 0; i <= n; i++ )
21             G[i].clear();
22         edges.clear();
23         m = 0;
24     }
25
26     void addedge( int u, int v, ll cap ) {
27         edges.emplace_back( v, cap );
28         edges.emplace_back( u, 0 );
29         G[u].push_back( m++ );
30         G[v].push_back( m++ );
31     }
32
33     bool BFS() {
34         memset( d, 0, sizeof d );
```

```

35     queue<int> Q;
36     Q.push( s );
37     d[s] = 1;
38     while ( !Q.empty() ) {
39         int x = Q.front();
40         Q.pop();
41         for ( int& i : G[x] ) {
42             E& e = edges[i];
43             if ( !d[e.to] && e.cp > 0 ) {
44                 d[e.to] = d[x] + 1;
45                 Q.push( e.to );
46             }
47         }
48     }
49     return d[t];
50 }
51
52 ll DFS( int u, ll cp ) {
53     if ( u == t || !cp )
54         return cp;
55     ll tmp = cp, f;
56     for ( int& i = cur[u]; i < G[u].size(); i++ ) {
57         E& e = edges[G[u][i]];
58         if ( d[u] + 1 == d[e.to] ) {
59             f = DFS( e.to, min( cp, e.cp ) );
60             e.cp -= f;
61             edges[G[u][i] ^ 1].cp += f;
62             cp -= f;
63             if ( !cp )
64                 break;
65         }
66     }
67     return tmp - cp;
68 }
69
70 ll go() {
71     ll flow = 0;
72     while ( BFS() ) {
73         memset( cur, 0, sizeof cur );
74         flow += DFS( s, INF );

```

```

75     }
76     return flow;
77 }
78 } DC;

```

4.2.2 费用流

```

1  constexpr ll INF = LLONG_MAX / 2;
2
3  struct E {
4      int from, to;
5      ll cp, v;
6      E() {}
7      E( int f, int t, ll cp, ll v ) : from( f ), to( t ), cp( cp ), v(
        v ) {}
8  };
9
10 struct MCMF {
11     static const int M = 1E5 * 5;
12     int n, m, s, t;
13     vector<E> edges;
14     vector<int> G[M];
15     bool inq[M];
16     ll d[M], a[M];
17     int p[M];
18
19     void init( int _n, int _s, int _t ) {
20         n = _n;
21         s = _s;
22         t = _t;
23         REP( i, 0, n + 1 ) G[i].clear();
24         edges.clear();
25         m = 0;
26     }
27
28     void addedge( int from, int to, ll cap, ll cost ) {
29         edges.emplace_back( from, to, cap, cost );
30         edges.emplace_back( to, from, 0, -cost );
31         G[from].push_back( m++ );
32         G[to].push_back( m++ );

```

```

33     }
34
35     bool BellmanFord( ll& flow, ll& cost ) {
36         REP( i, 0, n + 1 ) d[i] = INF;
37         memset( inq, 0, sizeof inq );
38         d[s] = 0, a[s] = INF, inq[s] = true;
39         queue<int> Q;
40         Q.push( s );
41         while ( !Q.empty() ) {
42             int u = Q.front();
43             Q.pop();
44             inq[u] = false;
45             for ( int& idx : G[u] ) {
46                 E& e = edges[idx];
47                 if ( e.cp && d[e.to] > d[u] + e.v ) {
48                     d[e.to] = d[u] + e.v;
49                     p[e.to] = idx;
50                     a[e.to] = min( a[u], e.cp );
51                     if ( !inq[e.to] ) {
52                         Q.push( e.to );
53                         inq[e.to] = true;
54                     }
55                 }
56             }
57         }
58         if ( d[t] == INF )
59             return false;
60         flow += a[t];
61         cost += a[t] * d[t];
62         int u = t;
63         while ( u != s ) {
64             edges[p[u]].cp -= a[t];
65             edges[p[u] ^ 1].cp += a[t];
66             u = edges[p[u]].from;
67         }
68         return true;
69     }
70
71     pair<ll, ll> go() {
72         ll flow = 0, cost = 0;

```

```

73     while ( BellmanFord( flow, cost ) )
74         ;
75     return make_pair( flow, cost );
76 }
77 } MM;

```

4.3 二分图最大匹配

ps. 建单向图（即只有左部指向右部的边）

```

1  struct MaxMatch {
2      int n;
3      vector<int> G[N];
4      int vis[N], left[N], clk;
5
6      void init( int n ) {
7          this->n = n;
8          REP( i, 0, n + 1 ) G[i].clear();
9          memset( left, -1, sizeof left );
10         memset( vis, -1, sizeof vis );
11     }
12
13     bool dfs( int u ) {
14         for ( int v : G[u] )
15             if ( vis[v] != clk ) {
16                 vis[v] = clk;
17                 if ( left[v] == -1 || dfs( left[v] ) ) {
18                     left[v] = u;
19                     return true;
20                 }
21             }
22         return false;
23     }
24
25     int match() {
26         int ret = 0;
27         for ( clk = 0; clk <= n; ++clk )
28             if ( dfs( clk ) )
29                 ++ret;
30         return ret;
31     }

```

```
32 } MM;
```

4.4 Tarjan 强连通分量缩点

```
1  int low[N], dfn[N], clk, B, bl[N];
2  vector<int> bcc[N];
3  void init() {
4      B = clk = 0;
5      memset( dfn, 0, sizeof dfn );
6  }
7  void tarjan( int u ) {
8      static int st[N], p;
9      static bool in[N];
10     dfn[u] = low[u] = ++clk;
11     st[p++] = u;
12     in[u] = true;
13     for ( int& v : G[u] ) {
14         if ( !dfn[v] ) {
15             tarjan( v );
16             low[u] = min( low[u], low[v] );
17         } else if ( in[v] )
18             low[u] = min( low[u], dfn[v] );
19     }
20     if ( dfn[u] == low[u] ) {
21         ++B;
22         while ( 1 ) {
23             int x = st[--p];
24             in[x] = false;
25             bl[x] = B;
26             bcc[B].push_back( x );
27             if ( x == u )
28                 break;
29         }
30     }
31 }
```

4.5 树直径

```
1  ll n, dep[N], mxdep[N];
2  vector<int> g[N];
```

```

3 vector<int> dmt;
4 void dfs1( int x, int fa ) {
5     dep[x] = dep[fa] + 1;
6     mxdep[x] = 1;
7     for ( int u : g[x] )
8         if ( u != fa ) {
9             dfs1( u, x );
10            mxdep[x] = max( mxdep[u] + 1, mxdep[x] );
11        }
12 }
13
14 void dfs2( int x, int fa ) { // 找一条直径，如果只需要直径长度则不
    用
15     dmt.push_back( x );
16     int nxt = -1;
17     for ( int u : g[x] )
18         if ( u != fa ) {
19             if ( nxt == -1 || mxdep[u] > mxdep[nxt] )
20                 nxt = u;
21         }
22     if ( nxt != -1 )
23         dfs2( nxt, x );
24 }
25
26 void diameter() {
27     dep[0] = 0;
28     dfs1( 1, 0 );
29     int rt = max_element( dep + 1, dep + n + 1 ) - dep;
30     dfs1( rt, 0 );
31     dfs2( rt, 0 );
32 }

```

4.6 树重心

```

1 ll n, sz[N], mxsz[N], G;
2 vector<int> g[N];
3 void dfs( int x, int fa ) {
4     sz[x] = 1, mxsz[x] = 0;
5     for ( int u : g[x] )
6         if ( u != fa ) {

```

```

7     dfs( u, x );
8     sz[x] += sz[u];
9     mxsz[x] = max( sz[u], mxsz[x] );
10    }
11    mxsz[x] = max( mxsz[x], n - sz[x] );
12    if ( G == -1 || ( mxsz[x] < mxsz[G] || ( mxsz[x] == mxsz[G] && x
        < G ) ) )
13        G = x;
14 }

```

4.7 树链剖分

```

1  ll fa[N], dep[N], son[N], sz[N], top[N], idx[N], clk, ridx[N], out[
    N];
2  vector<int> g[N];
3  void predfs( int x, int f ) { // 第一次 dfs 计算父亲/深度/大小/重儿
    子
4      fa[x] = f, dep[x] = dep[f] + 1, sz[x] = 1;
5      for ( auto v : g[x] )
6          if ( v != f ) {
7              predfs( v, x );
8              if ( sz[v] > sz[son[x]] )
9                  son[x] = v;
10                 sz[x] += sz[v];
11             }
12 }
13
14 void dfs( int x, int tp ) { // 第二次 dfs
15     // top 链的顶端
16     // idx 节点在 dfs 序中的编号
17     // ridx dfs 序中编号对应的节点
18     // out 以该节点为根的子树的编号的最大值
19     // 即 dfs 序中 [idx, out] 表示以它为根的子树
20     // [idx[top[x]], idx[x]] 表示这条链的顶端到该节点的一段链
21     top[x] = tp, idx[x] = ++clk, ridx[clk] = x;
22     if ( son[x] != 0 )
23         dfs( son[x], tp );
24     for ( auto v : g[x] )
25         if ( v != fa[x] && v != son[x] )
26             top[v] = v, dfs( v, v );

```

```

27     out[x] = clk;
28 }
29
30 void addpath( int u, int v, ll d ) {
31     int uu = top[u], vv = top[v];
32     while ( uu != vv ) {
33         if ( dep[uu] < dep[vv] )
34             swap( u, v ), swap( uu, vv );
35         add( 1, idx[uu], idx[u], d );
36         u = fa[uu], uu = top[u];
37     }
38     if ( dep[u] < dep[v] )
39         swap( u, v );
40     add( 1, idx[v], idx[u], d );
41 }
42
43 void addtree( int x, ll d ) {
44     add( 1, idx[x], out[x], d );
45 }
46
47 ll sumpath( int u, int v ) {
48     ll res = 0;
49     int uu = top[u], vv = top[v];
50     while ( uu != vv ) {
51         if ( dep[uu] < dep[vv] )
52             swap( u, v ), swap( uu, vv );
53         res = ( res + sum( 1, idx[uu], idx[u] ) ) % P;
54         u = fa[uu], uu = top[u];
55     }
56     if ( dep[u] < dep[v] )
57         swap( u, v );
58     res = ( res + sum( 1, idx[v], idx[u] ) ) % P;
59     return res;
60 }
61
62 ll sumtree( int x ) {
63     return sum( 1, idx[x], out[x] );
64 }

```

4.8 最短路

4.8.1 Floyd (最小环)

```
1 ll n, m, mincycle;
2 ll g[N][N], dis[N][N];
3 void floyd() {
4     // 如果 g[i][j] 之间没边则存 inf
5     // 注意 inf 的三倍不能爆 long long
6     mincycle = LLONG_MAX / 4;
7     REP( i, 1, n ) REP( j, 1, n ) dis[i][j] = g[i][j];
8     REP( k, 1, n ) {
9         REP( i, 1, k - 1 )
10            REP( j, i + 1, k - 1 ) {
11                mincycle = min( mincycle, dis[i][j] + g[j][k] + g[k][i] );
12            }
13         REP( i, 1, n )
14            REP( j, 1, n )
15                dis[i][j] = min( dis[i][j], dis[i][k] + dis[k][j] );
16     }
17 }
```

4.8.2 Spfa (判负环)

```
1 ll n, m, dis[N], inq[N], cnt[N];
2 vector<pair<int, ll>> g[N];
3 bool spfa( int s ) {
4     // 如果有负环则 return true
5     queue<int> q;
6     fill_n( dis + 1, n, LLONG_MAX / 2 );
7     fill_n( inq + 1, n, 0 );
8     fill_n( cnt + 1, n, 0 );
9     dis[s] = 0, inq[s] = cnt[s] = 1;
10    q.push( s );
11    while ( !q.empty() ) {
12        int u = q.front();
13        q.pop();
14        inq[u] = 0;
15        for ( auto [v, w] : g[u] ) {
16            if ( dis[v] > dis[u] + w ) {
17                dis[v] = dis[u] + w;
```

```

18         cnt[v] = cnt[u] + 1;
19         if ( cnt[v] > n )
20             return true;
21         if ( !inq[v] ) {
22             inq[v] = 1, q.push( v );
23         }
24     }
25 }
26 }
27 return false;
28 }

```

4.8.3 Dijkstra

```

1 ll n, m, dis[N], vis[N];
2 vector<pair<int, ll>> g[N];
3 void dijkstra( int s ) {
4     fill_n( dis + 1, n, LLONG_MAX / 2 );
5     fill_n( vis + 1, n, 0 );
6     dis[s] = 0;
7     priority_queue<pair<ll, int>> q;
8     q.push( make_pair( 0, s ) );
9     while ( !q.empty() ) {
10         int u = q.top().second;
11         q.pop();
12         if ( vis[u] )
13             continue;
14         vis[u] = 1;
15         for ( auto [v, w] : g[u] ) {
16             if ( vis[v] )
17                 continue;
18             if ( dis[v] > dis[u] + w ) {
19                 dis[v] = dis[u] + w;
20                 q.push( make_pair( -dis[v], v ) );
21             }
22         }
23     }
24 }

```

4.9 拓扑排序

```
1 ll n, in[N];
2 vector<int> g[N];
3 vector<int> res;
4 void topo() {
5     // 不要忘记在建图时记录入度
6     queue<int> q;
7     REP( i, 1, n )
8         if ( in[i] == 0 )
9             q.push( i );
10    while ( !q.empty() ) {
11        int u = q.front();
12        q.pop();
13        res.push_back( u );
14        for ( int v : g[u] ) {
15            --in[v];
16            if ( in[v] == 0 )
17                q.push( v );
18        }
19    }
20 }
```

4.10 最小生成树

Kruskal 算法

```
1 ll n, m, fa[N], weight_sum;
2 pair<ll, pair<int, int>> edge[M];
3 int find( int x ) {
4     if ( fa[x] == x )
5         return x;
6     return fa[x] = find( fa[x] );
7 }
8
9 void kruskal() {
10     weight_sum = 0;
11     sort( edge + 1, edge + m + 1 );
12     iota( fa + 1, fa + n + 1, 1 );
13     REP( i, 1, m ) {
14         auto [u, v] = edge[i].second;
```

```

15     u = find( u ), v = find( v );
16     if ( u == v )
17         continue;
18     fa[u] = v;
19     weight_sum += edge[i].first;
20 }
21 }

```

4.11 欧拉路径/回路

```

1 ll n, m, cnt[N];
2 vector<int> g[N];
3 vector<int> path;
4 void dfs( int x ) {
5     while ( cnt[x] < g[x].size() ) {
6         int u = g[x][cnt[x]];
7         ++cnt[x];
8         dfs( u );
9     }
10    path.push_back( x );
11    // 最后要 reverse(path.begin(), path.end());
12 }

```

5 字符串

5.1 KMP

```

1 char s1[N];
2 int n, pi[N];
3 void solve() {
4     // 字符串存于 s1[0, n)
5     pi[0] = 0;
6     REP( i, 1, n - 1 ) {
7         int j = pi[i - 1];
8         while ( j > 0 && s1[j] != s1[i] )
9             j = pi[j - 1];
10        if ( s1[j] == s1[i] )
11            ++j;
12        pi[i] = j;
13    }

```

```
14 }
```

5.2 Trie 树

```
1 int nxt[N][150], cnt;
2 int num[N];
3 void insert( char* s, int len ) {
4     // 字符串 s[0, len)
5     // 0 为根结点
6     int p = 0;
7     REP( i, 0, len - 1 ) {
8         if ( !nxt[p][s[i]] )
9             nxt[p][s[i]] = ++cnt;
10        p = nxt[p][s[i]];
11        ++num[p]; // 对前缀的统计在此处进行
12    }
13    // 对字符串整串的统计在此处进行
14 }
15
16 int find( char* s, int len ) {
17     int p = 0;
18     REP( i, 0, len - 1 ) {
19         if ( !nxt[p][s[i]] )
20             return 0;
21         p = nxt[p][s[i]];
22     }
23     return num[p];
24 }
```

5.3 Manacher

```
1 // s[0, n)
2 // d1[i] 表示 [i-d1[i], i+d1[i]] 是奇数长度回文 (i 为中心)
3 // d2[i] 表示 [i-d2[i]+1, i+d2[i]] 是偶数长度回文 (i+0.5 为中心)
4 // d2[i] 为 0 表示不存在
5
6 int l = 0, r = -1;
7 for ( int i = 0; i < n; ++i ) {
8     if ( r < i ) {
9         int j = i;
```

```

10     while ( j > 0 && 2 * i - ( j - 1 ) < n && s[j - 1] == s[2 * i -
11           ( j - 1 )] )
12         --j;
13     d1[i] = i - j;
14     l = j, r = 2 * i - j;
15 } else {
16     int j = ( l + r ) - i;
17     if ( j - d1[j] <= 1 ) {
18         int k = i * 2 - r;
19         while ( k > 0 && 2 * i - ( k - 1 ) < n
20               && s[k - 1] == s[2 * i - ( k - 1 )] )
21             --k;
22         d1[i] = i - k;
23         l = k, r = 2 * i - k;
24     } else
25         d1[i] = d1[j];
26 }
27
28 l = 0, r = -1;
29 for ( int i = 0; i + 1 < n; ++i ) {
30     if ( r < i ) {
31         int j = i;
32         while ( j >= 0 && i + i + 1 - j < n && s[j] == s[i + i + 1 - j]
33               )
34             --j;
35         d2[i] = i - j;
36         l = j + 1, r = i + i + 1 - j - 1;
37     } else {
38         int j = ( l + r ) - i - 1;
39         if ( j < 0 || j - d2[j] < 1 ) {
40             int k = i + i + 1 - r - 1;
41             while ( k >= 0 && i + i + 1 - k < n && s[k] == s[i + i + 1 -
42                   k] )
43                 --k;
44             d2[i] = i - k;
45             l = k + 1, r = i + i + 1 - k - 1;
46         } else
47             d2[i] = d2[j];
48     }
49 }

```

```
47 | }
```

上面的太丑了下面另一版

```
1 | int RL[N];
2 | // 最后结果的 RL 的说明:
3 | // 加了字符 '#' 的字符串中 [i - RL[i], i + RL[i]] 是极大回文串
4 | // 在原串中 RL[i] 正好是以 (i-1)/2 为中心极大回文串长度 (不论奇偶)
5 | void manacher( char* a, int n ) { // "abc" => "#a#b#c#"
6 |     int r = 0, p = 0;
7 |     REP( i, 0, n - 1 ) {
8 |         if ( i < r )
9 |             RL[i] = min( RL[2 * p - i], r - i );
10 |         // 上面这行 r-i 还是 r-i+1 结果一样, 尊重原作者
11 |         else
12 |             RL[i] = 1;
13 |         while ( i - RL[i] >= 0 && i + RL[i] < n && a[i - RL[i]] == a[i
14 |             + RL[i]] )
15 |             RL[i]++;
16 |         if ( RL[i] + i - 1 > r ) {
17 |             r = RL[i] + i - 1;
18 |             p = i;
19 |         }
20 |     }
21 |     REP( i, 0, n - 1 ) --RL[i];
22 | }
```

5.4 回文自动机

```
1 | // num 是该结点表示的前缀的回文后缀个数
2 | // cnt 是该结点表示的回文串在原串中的出现次数 (使用前需要向父亲更
3 |     新)
4 | namespace pam {
5 |
6 | int t[N][26], fa[N], len[N], rs[N], cnt[N], num[N];
7 | int sz, n, last;
8 | int _new( int l ) {
9 |     len[sz] = l;
10 |     cnt[sz] = num[sz] = 0;
11 |     return sz++;
12 | }
```

```

12 }
13 // 记得 init
14 void init() {
15     memset( t, 0, sz * sizeof t[0] );
16     rs[n = sz = 0] = -1;
17     last = _new( 0 );
18     fa[last] = _new( -1 );
19 }
20 int get_fa( int x ) {
21     while ( rs[n - 1 - len[x]] != rs[n] )
22         x = fa[x];
23     return x;
24 }
25 void ins( int ch ) {
26     rs[++n] = ch;
27     int p = get_fa( last );
28     if ( !t[p][ch] ) {
29         int np = _new( len[p] + 2 );
30         num[np] = num[fa[np] = t[get_fa( fa[p] )][ch]] + 1;
31         t[p][ch] = np;
32     }
33     ++cnt[last = t[p][ch]];
34 }
35
36 }

```

5.5 字符串哈希

```

1 #define ENABLE_DOUBLE_HASH
2
3 namespace hashstr {
4
5     typedef unsigned long long ull;
6     const int x = 135;
7     const int p1 = 0x3b3b3bbb, p2 = 1e9 + 1011;
8     ull xp1[N], xp2[N];
9
10    // 记得 init
11    void init_xp() {
12        xp1[0] = xp2[0] = 1;

```

```

13     for ( int i = 1; i < N; ++i ) {
14         xp1[i] = xp1[i - 1] * x % p1;
15         xp2[i] = xp2[i - 1] * x % p2;
16     }
17 }
18
19 struct str {
20     char s[N];
21     int length;
22     ull h[N];
23
24     ull hash() {
25         length = strlen( s );
26         ull res1 = 0, res2 = 0;
27         h[length] = 0;
28         for ( int j = length - 1; j >= 0; --j ) {
29 #ifdef ENABLE_DOUBLE_HASH
30             res1 = ( res1 * x + s[j] ) % p1;
31             res2 = ( res2 * x + s[j] ) % p2;
32             h[j] = ( res1 << 32 ) | res2;
33 #else
34             res1 = ( res1 * x + s[j] ) % p1;
35             h[j] = res1;
36 #endif
37         }
38         return h[0];
39     }
40     // 左闭右开区间
41     ull get_substr_hash( int left, int right ) const {
42         int len = right - left;
43 #ifdef ENABLE_DOUBLE_HASH
44         unsigned int mask32 = ~( 0u );
45         ull left1 = h[left] >> 32, right1 = h[right] >> 32;
46         ull left2 = h[left] & mask32, right2 = h[right] & mask32;
47         return ( ( ( left1 - right1 * xp1[len] % p1 + p1 ) % p1 ) << 32
48                 | ( ( ( left2 - right2 * xp2[len] % p2 + p2 ) % p2 ) ) );
49 #else
50         return ( ( h[left] - h[right] * xp1[len] % p1 + p1 ) % p1 );
51 #endif

```

```

52     }
53 };
54
55 }

```

5.6 后缀数组

```

1  int buf1[N], buf2[N], buc[N];
2
3  // str 存储在下标 [1, n] 的范围内, str[0] 要与 str[1..n] 都不相同
4  // ht[i] = LCP(sa[i], sa[i-1])
5  void sufsort( char str[], int n, int sa[], int rk[], int ht[] ) {
6      int *x = buf1, *y = buf2, m = 127;
7      REP( i, 0, m ) buc[i] = 0;
8      REP( i, 1, n ) buc[x[i]] = str[i]++;
9      REP( i, 1, m ) buc[i] += buc[i - 1];
10     REPd( i, n, 1 ) sa[buc[x[i]]--] = i;
11     for ( int k = 1; k <= n; k <= 1 ) {
12         int p = 0;
13         REP( i, n - k + 1, n ) y[++p] = i;
14         REP( i, 1, n ) if ( sa[i] > k ) y[++p] = sa[i] - k;
15         REP( i, 0, m ) buc[i] = 0;
16         REP( i, 1, n ) buc[x[y[i]]]++;
17         REP( i, 1, m ) buc[i] += buc[i - 1];
18         REPd( i, n, 1 ) sa[buc[x[y[i]]]--] = y[i];
19         swap( x, y ), x[sa[1]] = p = 1;
20         REP( i, 2, n )
21             if ( y[sa[i - 1]] == y[sa[i]] && y[sa[i - 1] + k] == y[sa[i] +
22                 k] )
23                 x[sa[i]] = p;
24             else
25                 x[sa[i]] = ++p;
26             if ( ( m = p ) >= n )
27                 break;
28     }
29     REP( i, 1, n ) rk[sa[i]] = i;
30     for ( int j = 0, k = 0, i = 1; i <= n; ht[rk[i++]] = k )
31         for ( k ? k-- : 0, j = sa[rk[i] - 1]; str[i + k] == str[j + k];
32             k++ )

```

32 | }

5.7 AC 自动机

```
1 // 洛谷模版题的代码为例
2
3 const int N = 2e6 + 100, M = 26;
4
5 int mp( char ch ) { return ch - 'a'; }
6
7 struct ACA {
8     int ch[N][M], fail[N];
9     int ridx[N], in[N];
10    int sz;
11    vector<int> topo; // 把拓扑序预处理出来 以便如果多次查询
12    void init() {
13        sz = 1;
14        memset( ch[0], 0, sizeof ch[0] );
15        in[0] = fail[0] = 0;
16    }
17    void insert( const string& s, int id ) {
18        int n = s.size();
19        int u = 0, c;
20        for ( int i = 0; i < n; ++i ) {
21            c = mp( s[i] );
22            if ( !ch[u][c] ) {
23                memset( ch[sz], 0, sizeof ch[sz] );
24                in[sz] = fail[sz] = 0;
25                ch[u][c] = sz++;
26            }
27            u = ch[u][c];
28        }
29        ridx[id] = u;
30    }
31    void build() {
32        queue<int> Q;
33        fail[0] = 0;
34        for ( int c = 0, u; c < M; c++ ) {
35            u = ch[0][c];
36            if ( u ) {
```

```

37         Q.push( u );
38         fail[u] = 0;
39         in[0]++;
40     }
41 }
42 while ( !Q.empty() ) {
43     int r = Q.front();
44     Q.pop();
45     for ( int c = 0, u; c < M; c++ ) {
46         u = ch[r][c];
47         if ( !u ) {
48             ch[r][c] = ch[fail[r]][c];
49             continue;
50         }
51         fail[u] = ch[fail[r]][c];
52         in[fail[u]]++;
53         Q.push( u );
54     }
55 }
56 }
57 void calc_topo() {
58     queue<int> Q;
59     for ( int i = 0; i < sz; ++i )
60         if ( in[i] == 0 )
61             Q.push( i );
62     while ( !Q.empty() ) {
63         int u = Q.front();
64         Q.pop();
65         topo.push_back( u );
66         if ( !u ) continue;
67         in[fail[u]]--;
68         if ( in[fail[u]] == 0 )
69             Q.push( fail[u] );
70     }
71 }
72 } ac;
73
74 int n;
75 char s[N];
76 int cnt[N];

```

```

77
78 int main() {
79     n = rd();
80     ac.init();
81     for ( int i = 1; i <= n; ++i ) {
82         scanf( "%s", s );
83         ac.insert( s, i );
84     }
85     ac.build();
86     ac.calc_topo();
87     scanf( "%s", s );
88     int u = 0, len = strlen( s );
89     for ( int i = 0; i < len; ++i ) {
90         u = ac.ch[u][mp( s[i] )];
91         ++cnt[u];
92     }
93     for ( int i : ac.topo )
94         cnt[ac.fail[i]] += cnt[i];
95     for ( int i = 1; i <= n; ++i )
96         printf( "%d\n", cnt[ac.riidx[i]] );
97     return 0;
98 }

```

6 计算几何

6.1 计算几何模板 - 吉如一

我也看不懂先放上来再说

```

1 #define mp make_pair
2 #define fi first
3 #define se second
4 #define pb push_back
5 typedef double db;
6 const db eps = 1e-6;
7 const db pi = acos(-1);
8 int sign(db k) {
9     if (k > eps) return 1; else if (k < -eps) return -1; return 0;
10 }
11 int cmp(db k1, db k2) {return sign(k1 - k2);}

```

```

12 int inmid(db k1, db k2, db k3) {return sign(k1 - k3) * sign(k2 - k3
    ) <= 0;} // k3 在 [k1,k2] 内
13 struct point {
14     db x, y;
15     point operator + (const point &k1) const {return (point) {k1.x +
        x, k1.y + y};}
16     point operator - (const point &k1) const {return (point) {x - k1.
        x, y - k1.y};}
17     point operator * (db k1) const {return (point) {x*k1, y*k1};}
18     point operator / (db k1) const {return (point) {x / k1, y / k1};}
19     int operator == (const point &k1) const {return cmp(x, k1.x) == 0
        && cmp(y, k1.y) == 0;}
20     // 逆时针旋转
21     point turn(db k1) {return (point) {x*cos(k1) - y*sin(k1), x*sin(
        k1) + y*cos(k1)};}
22     point turn90() {return (point) { -y, x};}
23     bool operator < (const point k1) const {
24         int a = cmp(x, k1.x);
25         if (a == -1) return 1; else if (a == 1) return 0; else return
            cmp(y, k1.y) == -1;
26     }
27     db abs() {return sqrt(x * x + y * y);}
28     db abs2() {return x * x + y * y;}
29     db dis(point k1) {return ((*this) - k1).abs();}
30     point unit() {db w = abs(); return (point) {x / w, y / w};}
31     void scan() {double k1, k2; scanf("%lf%lf", &k1, &k2); x = k1; y
        = k2;}
32     void print() {printf("%.11lf %.11lf\n", x, y);}
33     db getw() {return atan2(y, x);}
34     point getdel() {if (sign(x) == -1 || (sign(x) == 0 && sign(y) ==
        -1)) return (*this) * (-1); else return (*this);}
35     int getP() const {return sign(y) == 1 || (sign(y) == 0 && sign(x)
        == -1);}
36 };
37 int inmid(point k1, point k2, point k3) {return inmid(k1.x, k2.x,
    k3.x) && inmid(k1.y, k2.y, k3.y);}
38 db cross(point k1, point k2) {return k1.x * k2.y - k1.y * k2.x;}
39 db dot(point k1, point k2) {return k1.x * k2.x + k1.y * k2.y;}
40 db rad(point k1, point k2) {return atan2(cross(k1, k2), dot(k1, k2)
    );}

```

```

41 // -pi -> pi
42 int compareangle (point k1, point k2) {
43     return k1.getP() < k2.getP() || (k1.getP() == k2.getP() && sign(
        cross(k1, k2)) > 0);
44 }
45 point proj(point k1, point k2, point q) { // q 到直线 k1,k2 的投影
46     point k = k2 - k1; return k1 + k * (dot(q - k1, k) / k.abs2());
47 }
48 point reflect(point k1, point k2, point q) {return proj(k1, k2, q)
    * 2 - q;}
49 int clockwise(point k1, point k2, point k3) { // k1 k2 k3 逆时针 1
    顺时针 -1 否则 0
50     return sign(cross(k2 - k1, k3 - k1));
51 }
52 int checkLL(point k1, point k2, point k3, point k4) { // 求直线 (L)
    线段 (S)k1,k2 和 k3,k4 的交点
53     return cmp(cross(k3 - k1, k4 - k1), cross(k3 - k2, k4 - k2)) !=
        0;
54 }
55 point getLL(point k1, point k2, point k3, point k4) {
56     db w1 = cross(k1 - k3, k4 - k3), w2 = cross(k4 - k3, k2 - k3);
        return (k1 * w2 + k2 * w1) / (w1 + w2);
57 }
58 int intersect(db l1, db r1, db l2, db r2) {
59     if (l1 > r1) swap(l1, r1); if (l2 > r2) swap(l2, r2); return cmp(
        r1, l2) != -1 && cmp(r2, l1) != -1;
60 }
61 int checkSS(point k1, point k2, point k3, point k4) {
62     return intersect(k1.x, k2.x, k3.x, k4.x) && intersect(k1.y, k2.y,
        k3.y, k4.y) &&
63         sign(cross(k3 - k1, k4 - k1)) * sign(cross(k3 - k2, k4 -
            k2)) <= 0 &&
64         sign(cross(k1 - k3, k2 - k3)) * sign(cross(k1 - k4, k2 -
            k4)) <= 0;
65 }
66 db disSP(point k1, point k2, point q) {
67     point k3 = proj(k1, k2, q);
68     if (inmid(k1, k2, k3)) return q.dis(k3); else return min(q.dis(k1
        ), q.dis(k2));
69 }

```

```

70 db disSS(point k1, point k2, point k3, point k4) {
71     if (checkSS(k1, k2, k3, k4)) return 0;
72     else return min(min(disSP(k1, k2, k3), disSP(k1, k2, k4)), min(
        disSP(k3, k4, k1), disSP(k3, k4, k2)));
73 }
74 int onS(point k1, point k2, point q) {return inmid(k1, k2, q) &&
    sign(cross(k1 - q, k2 - k1)) == 0;}
75 struct circle {
76     point o; db r;
77     void scan() {o.scan(); scanf("%lf", &r);}
78     int inside(point k) {return cmp(r, o.dis(k));}
79 };
80 struct line {
81     // p[0]->p[1]
82     point p[2];
83     line(point k1, point k2) {p[0] = k1; p[1] = k2;}
84     point& operator [] (int k) {return p[k];}
85     int include(point k) {return sign(cross(p[1] - p[0], k - p[0])) >
        0;}
86     point dir() {return p[1] - p[0];}
87     line push() { // 向外 ( 左手边 ) 平移 eps
88         const db eps = 1e-6;
89         point delta = (p[1] - p[0]).turn90().unit() * eps;
90         return {p[0] - delta, p[1] - delta};
91     }
92 };
93 point getLL(line k1, line k2) {return getLL(k1[0], k1[1], k2[0], k2
    [1]);}
94 int parallel(line k1, line k2) {return sign(cross(k1.dir(), k2.dir
    ())) == 0;}
95 int sameDir(line k1, line k2) {return parallel(k1, k2) && sign(dot(
    k1.dir(), k2.dir())) == 1;}
96 int operator < (line k1, line k2) {
97     if (sameDir(k1, k2)) return k2.include(k1[0]);
98     return compareangle(k1.dir(), k2.dir());
99 }
100 int checkpos(line k1, line k2, line k3) {return k3.include(getLL(k1
    , k2));}
101 vector<line> getHL(vector<line> &L) { // 求半平面交 , 半平面是逆时
    针方向 , 输出按照逆时针

```

```

102 sort(L.begin(), L.end()); deque<line> q;
103 for (int i = 0; i < (int)L.size(); i++) {
104     if (i && sameDir(L[i], L[i - 1])) continue;
105     while (q.size() > 1 && !checkpos(q[q.size() - 2], q[q.size() -
106         1], L[i])) q.pop_back();
107     while (q.size() > 1 && !checkpos(q[1], q[0], L[i])) q.pop_front
108         ();
109     q.push_back(L[i]);
110 }
111 while (q.size() > 2 && !checkpos(q[q.size() - 2], q[q.size() -
112     1], q[0])) q.pop_back();
113 while (q.size() > 2 && !checkpos(q[1], q[0], q[q.size() - 1])) q.
114     pop_front();
115 vector<line>ans; for (int i = 0; i < q.size(); i++) ans.push_back
116     (q[i]);
117 return ans;
118 }
119 db closepoint(vector<point>&A, int l, int r) { // 最近点对 , 先要按
120     照 x 坐标排序
121     if (r - l <= 5) {
122         db ans = 1e20;
123         for (int i = l; i <= r; i++) for (int j = i + 1; j <= r; j++)
124             ans = min(ans, A[i].dis(A[j]));
125         return ans;
126     }
127     int mid = l + r >> 1; db ans = min(closepoint(A, l, mid),
128         closepoint(A, mid + 1, r));
129     vector<point>B; for (int i = l; i <= r; i++) if (abs(A[i].x - A[
130         mid].x) <= ans) B.push_back(A[i]);
131     sort(B.begin(), B.end(), [](point k1, point k2) {return k1.y < k2
132         .y;});
133     for (int i = 0; i < B.size(); i++) for (int j = i + 1; j < B.size
134         () && B[j].y - B[i].y < ans; j++) ans = min(ans, B[i].dis(B[j
135         ]));
136     return ans;
137 }
138 int checkposCC(circle k1, circle k2) { // 返回两个圆的公切线数量
139     if (cmp(k1.r, k2.r) == -1) swap(k1, k2);
140     db dis = k1.o.dis(k2.o); int w1 = cmp(dis, k1.r + k2.r), w2 =
141         cmp(dis, k1.r - k2.r);

```

```

129   if (w1 > 0) return 4; else if (w1 == 0) return 3; else if (w2 >
      0) return 2;
130   else if (w2 == 0) return 1; else return 0;
131 }
132 vector<point> getCL(circle k1, point k2, point k3) { // 沿着 k2->k3
      方向给出 , 相切给出两个
133   point k = proj(k2, k3, k1.o); db d = k1.r * k1.r - (k - k1.o).
      abs2();
134   if (sign(d) == -1) return {};
135   point del = (k3 - k2).unit() * sqrt(max((db)0.0, d)); return {k -
      del, k + del};
136 }
137 vector<point> getCC(circle k1, circle k2) { // 沿圆 k1 逆时针给出 ,
      相切给出两个
138   int pd = checkposCC(k1, k2); if (pd == 0 || pd == 4) return {};
139   db a = (k2.o - k1.o).abs2(), cosA = (k1.r * k1.r + a - k2.r * k2.
      r) / (2 * k1.r * sqrt(max(a, (db)0.0)));
140   db b = k1.r * cosA, c = sqrt(max((db)0.0, k1.r * k1.r - b * b));
141   point k = (k2.o - k1.o).unit(), m = k1.o + k * b, del = k.turn90
      () * c;
142   return {m - del, m + del};
143 }
144 vector<point> TangentCP(circle k1, point k2) { // 沿圆 k1 逆时针给
      出
145   db a = (k2 - k1.o).abs(), b = k1.r * k1.r / a, c = sqrt(max((db)
      0.0, k1.r * k1.r - b * b));
146   point k = (k2 - k1.o).unit(), m = k1.o + k * b, del = k.turn90()
      * c;
147   return {m - del, m + del};
148 }
149 vector<line> TangentoutCC(circle k1, circle k2) {
150   int pd = checkposCC(k1, k2); if (pd == 0) return {};
151   if (pd == 1) {point k = getCC(k1, k2)[0]; return {(line) {k, k
      }}};
152   if (cmp(k1.r, k2.r) == 0) {
153     point del = (k2.o - k1.o).unit().turn90().getdel();
154     return {(line){k1.o - del * k1.r, k2.o - del * k2.r}, (line) {
      k1.o + del*k1.r, k2.o + del*k2.r}};
155   } else {
156     point p = (k2.o * k1.r - k1.o * k2.r) / (k1.r - k2.r);

```

```

157     vector<point>A = TangentCP(k1, p), B = TangentCP(k2, p);
158     vector<line>ans; for (int i = 0; i < A.size(); i++) ans.
        push_back((line) {A[i], B[i]});
159     return ans;
160 }
161 }
162 vector<line> TangentinCC(circle k1, circle k2) {
163     int pd = checkposCC(k1, k2); if (pd <= 2) return {};
164     if (pd == 3) {point k = getCC(k1, k2)[0]; return {(line) {k, k
        }};};
165     point p = (k2.o * k1.r + k1.o * k2.r) / (k1.r + k2.r);
166     vector<point>A = TangentCP(k1, p), B = TangentCP(k2, p);
167     vector<line>ans; for (int i = 0; i < A.size(); i++) ans.push_back
        ((line) {A[i], B[i]});
168     return ans;
169 }
170 vector<line> TangentCC(circle k1, circle k2) {
171     int flag = 0; if (k1.r < k2.r) swap(k1, k2), flag = 1;
172     vector<line>A = TangentoutCC(k1, k2), B = TangentinCC(k1, k2);
173     for (line k : B) A.push_back(k);
174     if (flag) for (line &k : A) swap(k[0], k[1]);
175     return A;
176 }
177 db getarea(circle k1, point k2, point k3) {
178     // 圓 k1 與三角形 k2 k3 k1.o 的有向面積交
179     point k = k1.o; k1.o = k1.o - k; k2 = k2 - k; k3 = k3 - k;
180     int pd1 = k1.inside(k2), pd2 = k1.inside(k3);
181     vector<point>A = getCL(k1, k2, k3);
182     if (pd1 >= 0) {
183         if (pd2 >= 0) return cross(k2, k3) / 2;
184         return k1.r * k1.r * rad(A[1], k3) / 2 + cross(k2, A[1]) / 2;
185     } else if (pd2 >= 0) {
186         return k1.r * k1.r * rad(k2, A[0]) / 2 + cross(A[0], k3) / 2;
187     } else {
188         int pd = cmp(k1.r, disSP(k2, k3, k1.o));
189         if (pd <= 0) return k1.r * k1.r * rad(k2, k3) / 2;
190         return cross(A[0], A[1]) / 2 + k1.r * k1.r * (rad(k2, A[0]) +
            rad(A[1], k3)) / 2;
191     }
192 }

```

```

193 circle getcircle(point k1, point k2, point k3) {
194     db a1 = k2.x - k1.x, b1 = k2.y - k1.y, c1 = (a1 * a1 + b1 * b1) /
        2;
195     db a2 = k3.x - k1.x, b2 = k3.y - k1.y, c2 = (a2 * a2 + b2 * b2) /
        2;
196     db d = a1 * b2 - a2 * b1;
197     point o = (point) {k1.x + (c1 * b2 - c2 * b1) / d, k1.y + (a1 *
        c2 - a2 * c1) / d};
198     return (circle) {o, k1.dis(o)};
199 }
200 circle getScircle(vector<point> A) {
201     random_shuffle(A.begin(), A.end());
202     circle ans = (circle) {A[0], 0};
203     for (int i = 1; i < A.size(); i++)
204         if (ans.inside(A[i]) == -1) {
205             ans = (circle) {A[i], 0};
206             for (int j = 0; j < i; j++)
207                 if (ans.inside(A[j]) == -1) {
208                     ans.o = (A[i] + A[j]) / 2; ans.r = ans.o.dis(A[i]);
209                     for (int k = 0; k < j; k++)
210                         if (ans.inside(A[k]) == -1)
211                             ans = getcircle(A[i], A[j], A[k]);
212                 }
213         }
214     return ans;
215 }
216 db area(vector<point> A) { // 多边形用 vector<point> 表示 , 逆时针
217     db ans = 0;
218     for (int i = 0; i < A.size(); i++) ans += cross(A[i], A[(i + 1) %
        A.size()]);
219     return ans / 2;
220 }
221 int checkconvex(vector<point>A) {
222     int n = A.size(); A.push_back(A[0]); A.push_back(A[1]);
223     for (int i = 0; i < n; i++) if (sign(cross(A[i + 1] - A[i], A[i +
        2] - A[i])) == -1) return 0;
224     return 1;
225 }
226 int contain(vector<point>A, point q) { // 2 内部 1 边界 0 外部
227     int pd = 0; A.push_back(A[0]);

```

```

228     for (int i = 1; i < A.size(); i++) {
229         point u = A[i - 1], v = A[i];
230         if (onS(u, v, q)) return 1; if (cmp(u.y, v.y) > 0) swap(u, v);
231         if (cmp(u.y, q.y) >= 0 || cmp(v.y, q.y) < 0) continue;
232         if (sign(cross(u - v, q - v)) < 0) pd ^= 1;
233     }
234     return pd << 1;
235 }
236 vector<point> ConvexHull(vector<point>A, int flag = 1) { // flag=0
    不严格 flag=1 严格
237     int n = A.size(); vector<point>ans(n * 2);
238     sort(A.begin(), A.end()); int now = -1;
239     for (int i = 0; i < A.size(); i++) {
240         while (now > 0 && sign(cross(ans[now] - ans[now - 1], A[i] -
            ans[now - 1])) < flag) now--;
241         ans[++now] = A[i];
242     } int pre = now;
243     for (int i = n - 2; i >= 0; i--) {
244         while (now > pre && sign(cross(ans[now] - ans[now - 1], A[i] -
            ans[now - 1])) < flag) now--;
245         ans[++now] = A[i];
246     } ans.resize(now); return ans;
247 }
248 db convexDiameter(vector<point>A) {
249     int now = 0, n = A.size(); db ans = 0;
250     for (int i = 0; i < A.size(); i++) {
251         now = max(now, i);
252         while (1) {
253             db k1 = A[i].dis(A[now % n]), k2 = A[i].dis(A[(now + 1) % n])
                ;
254             ans = max(ans, max(k1, k2)); if (k2 > k1) now++; else break;
255         }
256     }
257     return ans;
258 }
259 vector<point> convexcut(vector<point>A, point k1, point k2) {
260     // 保留 k1,k2,p 逆时针的所有点
261     int n = A.size(); A.push_back(A[0]); vector<point>ans;
262     for (int i = 0; i < n; i++) {
263         int w1 = clockwise(k1, k2, A[i]), w2 = clockwise(k1, k2, A[i +

```

```

1]);
264     if (w1 >= 0) ans.push_back(A[i]);
265     if (w1 * w2 < 0) ans.push_back(getLL(k1, k2, A[i], A[i + 1]));
266 }
267 return ans;
268 }
269 int checkPoS(vector<point>A, point k1, point k2) {
270     // 多边形 A 和直线 ( 线段 )k1->k2 严格相交 , 注释部分为线段
271     struct ins {
272         point m, u, v;
273         int operator < (const ins& k) const {return m < k.m;}
274     }; vector<ins>B;
275     //if (contain(A,k1)==2||contain(A,k2)==2) return 1;
276     vector<point>poly = A; A.push_back(A[0]);
277     for (int i = 1; i < A.size(); i++) if (checkLL(A[i - 1], A[i], k1
278         , k2)) {
279         point m = getLL(A[i - 1], A[i], k1, k2);
280         if (inmid(A[i - 1], A[i], m)/*&&inmid(k1,k2,m)*/) B.push_back
281             ((ins) {m, A[i - 1], A[i]});
282     }
283     if (B.size() == 0) return 0; sort(B.begin(), B.end());
284     int now = 1; while (now < B.size() && B[now].m == B[0].m) now++;
285     if (now == B.size()) return 0;
286     int flag = contain(poly, (B[0].m + B[now].m) / 2);
287     if (flag == 2) return 1;
288     point d = B[now].m - B[0].m;
289     for (int i = now; i < B.size(); i++) {
290         if (!(B[i].m == B[i - 1].m) && flag == 2) return 1;
291         int tag = sign(cross(B[i].v - B[i].u, B[i].m + d - B[i].u));
292         if (B[i].m == B[i].u || B[i].m == B[i].v) flag += tag; else
293             flag += tag * 2;
294     }
295     //return 0;
296     return flag == 2;
297 }
298 int checkinp(point r, point l, point m) {
299     if (compareangle(l, r)) {return compareangle(l, m) &&
300         compareangle(m, r);}
301     return compareangle(l, m) || compareangle(m, r);
302 }

```

```

299 int checkPosFast(vector<point>A, point k1, point k2) { // 快速检查
    线段是否和多边形严格相交
300 if (contain(A, k1) == 2 || contain(A, k2) == 2) return 1; if (k1
    == k2) return 0;
301 A.push_back(A[0]); A.push_back(A[1]);
302 for (int i = 1; i + 1 < A.size(); i++)
303     if (checkLL(A[i - 1], A[i], k1, k2)) {
304         point now = getLL(A[i - 1], A[i], k1, k2);
305         if (inmid(A[i - 1], A[i], now) == 0 || inmid(k1, k2, now) ==
            0) continue;
306         if (now == A[i]) {
307             if (A[i] == k2) continue;
308             point pre = A[i - 1], ne = A[i + 1];
309             if (checkinp(pre - now, ne - now, k2 - now)) return 1;
310         } else if (now == k1) {
311             if (k1 == A[i - 1] || k1 == A[i]) continue;
312             if (checkinp(A[i - 1] - k1, A[i] - k1, k2 - k1)) return 1;
313         } else if (now == k2 || now == A[i - 1]) continue;
314         else return 1;
315     }
316 return 0;
317 }
318 // 拆分凸包成上下凸壳 凸包尽量都随机旋转一个角度来避免出现相同横坐
    标
319 // 尽量特判只有一个点的情况 凸包逆时针
320 void getUDP(vector<point>A, vector<point>&U, vector<point>&D) {
321     db l = 1e100, r = -1e100;
322     for (int i = 0; i < A.size(); i++) l = min(l, A[i].x), r = max(r,
        A[i].x);
323     int wherel, wherer;
324     for (int i = 0; i < A.size(); i++) if (cmp(A[i].x, l) == 0)
        wherel = i;
325     for (int i = A.size(); i; i--) if (cmp(A[i - 1].x, r) == 0)
        wherer = i - 1;
326     U.clear(); D.clear(); int now = wherel;
327     while (1) {D.push_back(A[now]); if (now == wherer) break; now++;
        if (now >= A.size()) now = 0;}
328     now = wherel;
329     while (1) {U.push_back(A[now]); if (now == wherer) break; now--;
        if (now < 0) now = A.size() - 1;}

```

```

330 }
331 // 需要保证凸包点数大于等于 3, 2 内部, 1 边界, 0 外部
332 int containCoP(const vector<point>&U, const vector<point>&D, point
    k) {
333     db lx = U[0].x, rx = U[U.size() - 1].x;
334     if (k == U[0] || k == U[U.size() - 1]) return 1;
335     if (cmp(k.x, lx) == -1 || cmp(k.x, rx) == 1) return 0;
336     int where1 = lower_bound(U.begin(), U.end(), (point) {k.x, -1e100
        }) - U.begin();
337     int where2 = lower_bound(D.begin(), D.end(), (point) {k.x, -1e100
        }) - D.begin();
338     int w1 = clockwise(U[where1 - 1], U[where1], k), w2 = clockwise(D
        [where2 - 1], D[where2], k);
339     if (w1 == 1 || w2 == -1) return 0; else if (w1 == 0 || w2 == 0)
        return 1; return 2;
340 }
341 // d 是方向, 输出上方切点和下方切点
342 pair<point, point> getTangentCow(const vector<point> &U, const
    vector<point> &D, point d) {
343     if (sign(d.x) < 0 || (sign(d.x) == 0 && sign(d.y) < 0)) d = d *
        (-1);
344     point whereU, whereD;
345     if (sign(d.x) == 0) return mp(U[0], U[U.size() - 1]);
346     int l = 0, r = U.size() - 1, ans = 0;
347     while (l < r) {int mid = l + r >> 1; if (sign(cross(U[mid + 1] -
        U[mid], d)) <= 0) l = mid + 1, ans = mid + 1; else r = mid;}
348     whereU = U[ans]; l = 0, r = D.size() - 1, ans = 0;
349     while (l < r) {int mid = l + r >> 1; if (sign(cross(D[mid + 1] -
        D[mid], d)) >= 0) l = mid + 1, ans = mid + 1; else r = mid;}
350     whereD = D[ans]; return mp(whereU, whereD);
351 }
352 // 先检查 contain, 逆时针给出
353 pair<point, point> getTangentCoP(const vector<point>&U, const
    vector<point>&D, point k) {
354     db lx = U[0].x, rx = U[U.size() - 1].x;
355     if (k.x < lx) {
356         int l = 0, r = U.size() - 1, ans = U.size() - 1;
357         while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid], U
            [mid + 1]) == 1) l = mid + 1; else ans = mid, r = mid;}
358         point w1 = U[ans]; l = 0, r = D.size() - 1, ans = D.size() - 1;

```

```

359     while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid], D
        [mid + 1]) == -1) l = mid + 1; else ans = mid, r = mid;}
360     point w2 = D[ans]; return mp(w1, w2);
361 } else if (k.x > rx) {
362     int l = 1, r = U.size(), ans = 0;
363     while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid], U
        [mid - 1]) == -1) r = mid; else ans = mid, l = mid + 1;}
364     point w1 = U[ans]; l = 1, r = D.size(), ans = 0;
365     while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid], D
        [mid - 1]) == 1) r = mid; else ans = mid, l = mid + 1;}
366     point w2 = D[ans]; return mp(w2, w1);
367 } else {
368     int where1 = lower_bound(U.begin(), U.end(), (point) {k.x, -1
        e100}) - U.begin();
369     int where2 = lower_bound(D.begin(), D.end(), (point) {k.x, -1
        e100}) - D.begin();
370     if ((k.x == lx && k.y > U[0].y) || (where1 && clockwise(U[
        where1 - 1], U[where1], k) == 1)) {
371         int l = 1, r = where1 + 1, ans = 0;
372         while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid],
            U[mid - 1]) == 1) ans = mid, l = mid + 1; else r = mid;}
373         point w1 = U[ans]; l = where1, r = U.size() - 1, ans = U.size
            () - 1;
374         while (l < r) {int mid = l + r >> 1; if (clockwise(k, U[mid],
            U[mid + 1]) == 1) l = mid + 1; else ans = mid, r = mid;}
375         point w2 = U[ans]; return mp(w2, w1);
376     } else {
377         int l = 1, r = where2 + 1, ans = 0;
378         while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid],
            D[mid - 1]) == -1) ans = mid, l = mid + 1; else r = mid;}
379         point w1 = D[ans]; l = where2, r = D.size() - 1, ans = D.size
            () - 1;
380         while (l < r) {int mid = l + r >> 1; if (clockwise(k, D[mid],
            D[mid + 1]) == -1) l = mid + 1; else ans = mid, r = mid;}
381         point w2 = D[ans]; return mp(w1, w2);
382     }
383 }
384 }
385 struct P3 {
386     db x, y, z;

```

```

387 P3 operator + (P3 k1) {return (P3) {x + k1.x, y + k1.y, z + k1.z
    };}
388 P3 operator - (P3 k1) {return (P3) {x - k1.x, y - k1.y, z - k1.z
    };}
389 P3 operator * (db k1) {return (P3) {x*k1, y*k1, z*k1};}
390 P3 operator / (db k1) {return (P3) {x / k1, y / k1, z / k1};}
391 db abs2() {return x * x + y * y + z * z;}
392 db abs() {return sqrt(x * x + y * y + z * z);}
393 P3 unit() {return (*this) / abs();}
394 int operator < (const P3 k1) const {
395     if (cmp(x, k1.x) != 0) return x < k1.x;
396     if (cmp(y, k1.y) != 0) return y < k1.y;
397     return cmp(z, k1.z) == -1;
398 }
399 int operator == (const P3 k1) {
400     return cmp(x, k1.x) == 0 && cmp(y, k1.y) == 0 && cmp(z, k1.z)
        == 0;
401 }
402 void scan() {
403     double k1, k2, k3; scanf("%lf%lf%lf", &k1, &k2, &k3);
404     x = k1; y = k2; z = k3;
405 }
406 };
407 P3 cross(P3 k1, P3 k2) {return (P3) {k1.y*k2.z - k1.z*k2.y, k1.z*k2
    .x - k1.x*k2.z, k1.x*k2.y - k1.y*k2.x};}
408 db dot(P3 k1, P3 k2) {return k1.x * k2.x + k1.y * k2.y + k1.z * k2.
    z;}
409 //p=(3,4,5),l=(13,19,21),theta=85 ans=(2.83,4.62,1.77)
410 P3 turn3D(db k1, P3 l, P3 p) {
411     l = l.unit(); P3 ans; db c = cos(k1), s = sin(k1);
412     ans.x = p.x * (l.x * l.x * (1 - c) + c) + p.y * (l.x * l.y * (1 -
        c) - l.z * s) + p.z * (l.x * l.z * (1 - c) + l.y * s);
413     ans.y = p.x * (l.x * l.y * (1 - c) + l.z * s) + p.y * (l.y * l.y
        * (1 - c) + c) + p.z * (l.y * l.z * (1 - c) - l.x * s);
414     ans.z = p.x * (l.x * l.z * (1 - c) - l.y * s) + p.y * (l.y * l.z
        * (1 - c) + l.x * s) + p.z * (l.x * l.x * (1 - c) + c);
415     return ans;
416 }
417 typedef vector<P3> VP;
418 typedef vector<VP> VVP;

```

```

419 db Acos(db x) {return acos(max(-(db)1, min(x, (db)1)));}
420 // 球面距离 , 圆心原点 , 半径 1
421 db Odist(P3 a, P3 b) {db r = Acos(dot(a, b)); return r;}
422 db r; P3 rnd;
423 vector<db> solve(db a, db b, db c) {
424     db r = sqrt(a * a + b * b), th = atan2(b, a);
425     if (cmp(c, -r) == -1) return {0};
426     else if (cmp(r, c) <= 0) return {1};
427     else {
428         db tr = pi - Acos(c / r); return {th + pi - tr, th + pi + tr};
429     }
430 }
431 vector<db> jiao(P3 a, P3 b) {
432     // dot(rd+x*cos(t)+y*sin(t),b) >= cos(r)
433     if (cmp(Odist(a, b), 2 * r) > 0) return {0};
434     P3 rd = a * cos(r), z = a.unit(), y = cross(z, rnd).unit(), x =
        cross(y, z).unit();
435     vector<db> ret = solve(-(dot(x, b) * sin(r)), -(dot(y, b) * sin(r)
        )), -(cos(r) - dot(rd, b)));
436     return ret;
437 }
438 db norm(db x, db l = 0, db r = 2 * pi) { // change x into [l,r)
439     while (cmp(x, l) == -1) x += (r - l); while (cmp(x, r) >= 0) x -=
        (r - l);
440     return x;
441 }
442 db disLP(P3 k1, P3 k2, P3 q) {
443     return (cross(k2 - k1, q - k1)).abs() / (k2 - k1).abs();
444 }
445 db disLL(P3 k1, P3 k2, P3 k3, P3 k4) {
446     P3 dir = cross(k2 - k1, k4 - k3); if (sign(dir.abs()) == 0)
        return disLP(k1, k2, k3);
447     return fabs(dot(dir.unit(), k1 - k2));
448 }
449 VP getFL(P3 p, P3 dir, P3 k1, P3 k2) {
450     db a = dot(k2 - p, dir), b = dot(k1 - p, dir), d = a - b;
451     if (sign(fabs(d)) == 0) return {};
452     return {(k1 * a - k2 * b) / d};
453 }
454 VP getFF(P3 p1, P3 dir1, P3 p2, P3 dir2) { // 返回一条线

```

```

455     P3 e = cross(dir1, dir2), v = cross(dir1, e);
456     db d = dot(dir2, v); if (sign(abs(d)) == 0) return {};
457     P3 q = p1 + v * dot(dir2, p2 - p1) / d; return {q, q + e};
458 }
459 // 3D Covex Hull Template
460 db getV(P3 k1, P3 k2, P3 k3, P3 k4) { // get the Volume
461     return dot(cross(k2 - k1, k3 - k1), k4 - k1);
462 }
463 db rand_db() {return 1.0 * rand() / RAND_MAX;}
464 VP convexHull2D(VP A, P3 dir) {
465     P3 x = {(db)rand(), (db)rand(), (db)rand()}; x = x.unit();
466     x = cross(x, dir).unit(); P3 y = cross(x, dir).unit();
467     P3 vec = dir.unit() * dot(A[0], dir);
468     vector<point>B;
469     for (int i = 0; i < A.size(); i++) B.push_back((point) {dot(A[i],
470         x), dot(A[i], y)});
471     B = ConvexHull(B); A.clear();
472     for (int i = 0; i < B.size(); i++) A.push_back(x * B[i].x + y * B
473         [i].y + vec);
474     return A;
475 }
476 namespace CH3 {
477 VVP ret; set<pair<int, int> >e;
478 int n; VP p, q;
479 void wrap(int a, int b) {
480     if (e.find({a, b}) == e.end()) {
481         int c = -1;
482         for (int i = 0; i < n; i++) if (i != a && i != b) {
483             if (c == -1 || sign(getV(q[c], q[a], q[b], q[i])) > 0) c =
484                 i;
485         }
486         if (c != -1) {
487             ret.push_back({p[a], p[b], p[c]});
488             e.insert({a, b}); e.insert({b, c}); e.insert({c, a});
489             wrap(c, b); wrap(a, c);
490         }
491     }
492 }
493 VVP ConvexHull3D(VP _p) {
494     p = q = _p; n = p.size();

```

```

492   ret.clear(); e.clear();
493   for (auto &i : q) i = i + (P3) {rand_db() * 1e-4, rand_db() * 1e
      -4, rand_db() * 1e-4};
494   for (int i = 1; i < n; i++) if (q[i].x < q[0].x) swap(p[0], p[i])
      , swap(q[0], q[i]);
495   for (int i = 2; i < n; i++) if ((q[i].x - q[0].x) * (q[1].y - q
      [0].y) > (q[i].y - q[0].y) * (q[1].x - q[0].x)) swap(q[1], q[i]
      ), swap(p[1], p[i]);
496   wrap(0, 1);
497   return ret;
498 }
499 }
500 VVP reduceCH(VVP A) {
501   VVP ret; map<P3, VP> M;
502   for (VP nowF : A) {
503     P3 dir = cross(nowF[1] - nowF[0], nowF[2] - nowF[0]).unit();
504     for (P3 k1 : nowF) M[dir].pb(k1);
505   }
506   for (pair<P3, VP> nowF : M) ret.pb(convexHull2D(nowF.se, nowF.fi)
      );
507   return ret;
508 }
509 // 把一个面变成 ( 点 , 法向量 ) 的形式
510 pair<P3, P3> getF(VP F) {
511   return mp(F[0], cross(F[1] - F[0], F[2] - F[0]).unit());
512 }
513 // 3D Cut 保留 dot(dir,x-p)>=0 的部分
514 VVP ConvexCut3D(VVP A, P3 p, P3 dir) {
515   VVP ret; VP sec;
516   for (VP nowF : A) {
517     int n = nowF.size(); VP ans; int dif = 0;
518     for (int i = 0; i < n; i++) {
519       int d1 = sign(dot(dir, nowF[i] - p));
520       int d2 = sign(dot(dir, nowF[(i + 1) % n] - p));
521       if (d1 >= 0) ans.pb(nowF[i]);
522       if (d1 * d2 < 0) {
523         P3 q = getFL(p, dir, nowF[i], nowF[(i + 1) % n])[0];
524         ans.push_back(q); sec.push_back(q);
525       }
526       if (d1 == 0) sec.push_back(nowF[i]); else dif = 1;

```

```

527         dif |= (sign(dot(dir, cross(nowF[(i + 1) % n] - nowF[i], nowF
           [(i + 1) % n] - nowF[i]))) == -1);
528     }
529     if (ans.size() > 0 && dif) ret.push_back(ans);
530 }
531 if (sec.size() > 0) ret.push_back(convexHull2D(sec, dir));
532 return ret;
533 }
534 db vol(VVP A) {
535     if (A.size() == 0) return 0; P3 p = A[0][0]; db ans = 0;
536     for (VP nowF : A)
537         for (int i = 2; i < nowF.size(); i++)
538             ans += abs(getV(p, nowF[0], nowF[i - 1], nowF[i]));
539     return ans / 6;
540 }
541 VVP init(db INF) {
542     VVP pss(6, VP(4));
543     pss[0][0] = pss[1][0] = pss[2][0] = { -INF, -INF, -INF};
544     pss[0][3] = pss[1][1] = pss[5][2] = { -INF, -INF, INF};
545     pss[0][1] = pss[2][3] = pss[4][2] = { -INF, INF, -INF};
546     pss[0][2] = pss[5][3] = pss[4][1] = { -INF, INF, INF};
547     pss[1][3] = pss[2][1] = pss[3][2] = {INF, -INF, -INF};
548     pss[1][2] = pss[5][1] = pss[3][3] = {INF, -INF, INF};
549     pss[2][2] = pss[4][3] = pss[3][1] = {INF, INF, -INF};
550     pss[5][0] = pss[4][0] = pss[3][0] = {INF, INF, INF};
551     return pss;
552 }

```

7 STL

7.1 算法库

不修改序列的操作

批量操作

在标头 `<algorithm>` 定义

`for_each`

应用一元函数对象到范围中元素 (函数模板)

`ranges::for_each` (C++20)

应用一元函数对象到范围中元素 (算法函数对象)

`for_each_n` (C++17)

应用函数对象到序列的前 N 个元素 (函数模板)

`ranges::for_each_n` (C++20)

应用函数对象到序列的前 N 个元素 (算法函数对象)

搜索操作

在标头 `<algorithm>` 定义

`all_of` (C++11)

`any_of` (C++11)

`none_of` (C++11)

检查谓词是否对范围中所有、任一或无元素为 `true` (函数模板)

`ranges::all_of` (C++20)

`ranges::any_of` (C++20)

`ranges::none_of` (C++20)

检查谓词是否对范围中所有、任一或无元素为 `true` (算法函数对象)

`ranges::contains` (C++23)

`ranges::contains_subrange` (C++23)

检查范围是否包含给定元素或子范围 (算法函数对象)

`find`

`find_if`

`find_if_not` (C++11)

查找首个满足特定条件的元素 (函数模板)

`ranges::find` (C++20)

`ranges::find_if` (C++20)

`ranges::find_if_not` (C++20)

查找首个满足特定条件的元素 (算法函数对象)

`ranges::find_last` (C++23)

`ranges::find_last_if` (C++23)

`ranges::find_last_if_not` (C++23)

查找最后一个满足特定条件的元素 (算法函数对象)

`find_end`

查找元素序列在特定范围中最后一次出现 (函数模板)

`ranges::find_end` (C++20)

查找元素序列在特定范围中最后一次出现 (算法函数对象)

`find_first_of`

搜索一组元素中任一元素 (函数模板)

`ranges::find_first_of` (C++20)

搜索一组元素中任一元素 (算法函数对象)

`adjacent_find`

查找首对相同 (或满足给定谓词) 的相邻元素 (函数模板)

`ranges::adjacent_find` (C++20)

查找首对相同 (或满足给定谓词) 的相邻元素 (算法函数对象)

count
count_if
 返回满足特定条件的元素数目 (函数模板)
ranges::count (C++20)
ranges::count_if (C++20)
 返回满足特定条件的元素数目 (算法函数对象)
mismatch
 查找两个范围的首个不同之处 (函数模板)
ranges::mismatch (C++20)
 查找两个范围的首个不同之处 (算法函数对象)
equal
 判断两组元素是否相同 (函数模板)
ranges::equal (C++20)
 判断两组元素是否相同 (算法函数对象)
search
 搜索元素范围的首次出现 (函数模板)
ranges::search (C++20)
 搜索元素范围的首次出现 (算法函数对象)
search_n
 搜索元素在范围中首次连续若干次出现 (函数模板)
ranges::search_n (C++20)
 搜索元素在范围中首次连续若干次出现 (算法函数对象)
ranges::starts_with (C++23)
 检查一个范围是否始于另一范围 (算法函数对象)
ranges::ends_with (C++23)
 检查一个范围是否终于另一范围 (算法函数对象)
折叠操作 (C++23 起)
 在标头 `<algorithm>` 定义
ranges::fold_left (C++23)
 左折叠范围中元素 (算法函数对象)
ranges::fold_left_first (C++23)
 以首元素为初值左折叠范围中元素 (算法函数对象)
ranges::fold_right (C++23)
 右折叠范围中元素 (算法函数对象)
ranges::fold_right_last (C++23)
 以末元素为初值右折叠范围中元素 (算法函数对象)
ranges::fold_left_with_iter (C++23)
 左折叠范围中元素, 并返回 **pair** (迭代器, 值) (算法函数对象)
ranges::fold_left_first_with_iter (C++23)
 以首元素为初值左折叠范围中元素, 并返回 **pair** (迭代器, optional) (算法函数对象)

修改序列的操作

复制操作

在标头 `<algorithm>` 定义

`copy`

`copy_if` (C++11)

复制范围中元素到新位置 (函数模板)

`ranges::copy` (C++20)

`ranges::copy_if` (C++20)

复制范围中元素到新位置 (算法函数对象)

`copy_n` (C++11)

复制若干元素到新位置 (函数模板)

`ranges::copy_n` (C++20)

复制若干元素到新位置 (算法函数对象)

`copy_backward`

从后往前复制范围中元素 (函数模板)

`ranges::copy_backward` (C++20)

从后往前复制范围中元素 (算法函数对象)

`move` (C++11)

将范围中元素移到新位置 (函数模板)

`ranges::move` (C++20)

将范围中元素移到新位置 (算法函数对象)

`move_backward` (C++11)

从后往前将范围中元素移到新位置 (函数模板)

`ranges::move_backward` (C++20)

从后往前将范围中元素移到新位置 (算法函数对象)

交换操作

在标头 `<algorithm>` 定义 (C++11 前)

在标头 `<utility>` 定义 (C++11 起)

在标头 `<string_view>` 定义

`swap`

在标头 `<algorithm>` 定义

交换两个对象的值 (函数模板)

`swap_ranges`

交换两个范围的元素 (函数模板)

`ranges::swap_ranges` (C++20)

交换两个范围的元素 (算法函数对象)

`iter_swap`

交换两个迭代器所指向的元素 (函数模板)

变换操作

在标头 `<algorithm>` 定义

transform

应用函数到元素范围，并在目标范围存储结果 (函数模板)

ranges::transform (C++20)

应用函数到元素范围 (算法函数对象)

replace

replace_if

替换所有满足特定条件的值为另一个值 (函数模板)

ranges::replace (C++20)

ranges::replace_if (C++20)

替换所有满足特定条件的值为另一个值 (算法函数对象)

replace_copy

replace_copy_if

复制范围，并将满足特定条件的元素替换为另一个值 (函数模板)

ranges::replace_copy (C++20)

ranges::replace_copy_if (C++20)

复制范围，并将满足特定条件的元素替换为另一个值 (算法函数对象)

生成操作

在标头 `<algorithm>` 定义

fill

以复制的方式赋给定值到范围中所有元素 (函数模板)

ranges::fill (C++20)

赋给定值到范围中元素 (算法函数对象)

fill_n

以复制的方式赋给定值到范围中 N 个元素 (函数模板)

ranges::fill_n (C++20)

赋给定值到若干元素 (算法函数对象)

generate

赋连续函数调用结果到范围中所有元素 (函数模板)

ranges::generate (C++20)

将函数结果保存到范围中 (算法函数对象)

generate_n

赋连续函数调用结果到范围中 N 个元素 (函数模板)

ranges::generate_n (C++20)

保存 N 次函数应用的结果 (算法函数对象)

移除操作

在标头 `<algorithm>` 定义

remove

remove_if

移除满足特定条件的元素 (函数模板)

ranges::remove (C++20)

ranges::remove_if (C++20)
 移除满足特定条件的元素 (算法函数对象)

remove_copy

remove_copy_if
 复制范围并忽略满足特定条件的元素 (函数模板)

ranges::remove_copy (C++20)

ranges::remove_copy_if (C++20)
 复制范围并忽略满足特定条件的元素 (算法函数对象)

unique
 移除范围中连续重复元素 (函数模板)

ranges::unique (C++20)
 移除范围中连续重复元素 (算法函数对象)

unique_copy
 创建某范围的不含连续重复元素的副本 (函数模板)

ranges::unique_copy (C++20)
 创建某范围的不含连续重复元素的副本 (算法函数对象)

顺序变更操作
 在标头 `<algorithm>` 定义

reverse
 逆转范围中的元素顺序 (函数模板)

ranges::reverse (C++20)
 逆转范围中的元素顺序 (算法函数对象)

reverse_copy
 创建范围的逆向副本 (函数模板)

ranges::reverse_copy (C++20)
 创建范围的逆向副本 (算法函数对象)

rotate
 旋转范围中的元素顺序 (函数模板)

ranges::rotate (C++20)
 旋转范围中的元素顺序 (算法函数对象)

rotate_copy
 复制并旋转元素范围 (函数模板)

ranges::rotate_copy (C++20)
 复制并旋转元素范围 (算法函数对象)

shift_left (C++20)

shift_right (C++20)
 迁移范围中元素 (函数模板)

ranges::shift_left (C++23)

ranges::shift_right (C++23)
 迁移范围中元素 (算法函数对象)

`random_shuffle` (C++17 前)

`shuffle` (C++11)

随机重排范围中元素 (函数模板)

`ranges::shuffle` (C++20)

随机重排范围中元素 (算法函数对象)

采样操作

在标头 `<algorithm>` 定义

`sample` (C++17)

从序列中随机选择 N 个元素 (函数模板)

`ranges::sample` (C++20)

从序列中随机选择 N 个元素 (算法函数对象)

排序和相关操作

要求

部分算法要求由实参表示的序列“已排序”或“已划分”。未满足要求时行为未定义。

序列 `[start, finish)` 在满足以下条件时已按表达式 `f(e)` 划分: 存在一个整数 `n`, 使得对于 `[0, std::distance(start, finish))` 中的所有整数 `i`, `f(*(start + i))` 当且仅当 `i < n` 时是 `true`。

划分操作

在标头 `<algorithm>` 定义

`is_partitioned` (C++11)

判断范围是否已按给定谓词划分 (函数模板)

`ranges::is_partitioned` (C++20)

判断范围是否已按给定谓词划分 (算法函数对象)

`partition`

将范围中元素分为两组 (函数模板)

`ranges::partition` (C++20)

将范围中元素分为两组 (算法函数对象)

`partition_copy` (C++11)

复制范围并将元素分为两组 (函数模板)

`ranges::partition_copy` (C++20)

复制范围并将元素分为两组 (算法函数对象)

`stable_partition`

将元素分为两组, 同时保留其相对顺序 (函数模板)

`ranges::stable_partition` (C++20)

将元素分为两组, 同时保留其相对顺序 (算法函数对象)

`partition_point` (C++11)

定位已划分范围的划分点 (函数模板)

`ranges::partition_point` (C++20)

定位已划分范围的划分点 (算法函数对象)

排序操作

在标头 `<algorithm>` 定义

sort

将范围按升序排序 (函数模板)

ranges::sort (C++20)

将范围按升序排序 (算法函数对象)

stable_sort

将范围中元素排序, 同时保持相等元之间的顺序 (函数模板)

ranges::stable_sort (C++20)

将范围中元素排序, 同时保持相等元之间的顺序 (算法函数对象)

partial_sort

将范围中前 N 个元素排序 (函数模板)

ranges::partial_sort (C++20)

将范围中前 N 个元素排序 (算法函数对象)

partial_sort_copy

复制范围中元素并部分排序 (函数模板)

ranges::partial_sort_copy (C++20)

复制范围中元素并部分排序 (算法函数对象)

is_sorted (C++11)

检查范围是否已按升序排列 (函数模板)

ranges::is_sorted (C++20)

检查范围是否已按升序排列 (算法函数对象)

is_sorted_until (C++11)

找出最大的有序子范围 (函数模板)

ranges::is_sorted_until (C++20)

找出最大的有序子范围 (算法函数对象)

nth_element

将给定范围部分排序, 确保其按给定元素划分 (函数模板)

ranges::nth_element (C++20)

将给定范围部分排序, 确保其按给定元素划分 (算法函数对象)

二分搜索操作 (在已划分范围上)

在标头 `<algorithm>` 定义

lower_bound

返回首个不小于给定值的元素的迭代器 (函数模板)

ranges::lower_bound (C++20)

返回首个不小于给定值的元素的迭代器 (算法函数对象)

upper_bound

返回首个大于给定值的元素的迭代器 (函数模板)

ranges::upper_bound (C++20)

返回首个大于给定值的元素的迭代器 (算法函数对象)

equal_range

返回匹配特定键值的元素范围 (函数模板)

`ranges::equal_range` (C++20)

返回匹配特定键值的元素范围 (算法函数对象)

`binary_search`

判断元素是否在偏序范围中 (函数模板)

`ranges::binary_search` (C++20)

判断元素是否在偏序范围中 (算法函数对象)

集合操作 (在已排序范围上)

在标头 `<algorithm>` 定义

`includes`

当一个序列是另一个的子序列时返回 `true` (函数模板)

`ranges::includes` (C++20)

当一个序列是另一个的子序列时返回 `true` (算法函数对象)

`set_union`

计算两个集合的并集 (函数模板)

`ranges::set_union` (C++20)

计算两个集合的并集 (算法函数对象)

`set_intersection`

计算两个集合的交集 (函数模板)

`ranges::set_intersection` (C++20)

计算两个集合的交集 (算法函数对象)

`set_difference`

计算两个集合的差集 (函数模板)

`ranges::set_difference` (C++20)

计算两个集合的差集 (算法函数对象)

`set_symmetric_difference`

计算两个集合的对称差 (函数模板)

`ranges::set_symmetric_difference` (C++20)

计算两个集合的对称差 (算法函数对象)

归并操作 (在已排序范围上)

在标头 `<algorithm>` 定义

`merge`

合并两个有序范围 (函数模板)

`ranges::merge` (C++20)

合并两个有序范围 (算法函数对象)

`inplace_merge`

就地合并两个有序范围 (函数模板)

`ranges::inplace_merge` (C++20)

就地合并两个有序范围 (算法函数对象)

堆操作

(C++20 前) 随机访问范围 `[first, last)` 在满足以下条件时是一个关于比较器 `comp` 的堆: 对于 $(0, last - first)$ 中的所有整数 i , `bool(comp(first[(i - 1) / 2], first[i]))` 都是 `false`。

在标头 `<algorithm>` 定义

push_heap

添加元素到最大堆 (函数模板)

ranges::push_heap (C++20)

添加元素到最大堆 (算法函数对象)

pop_heap

移除最大堆中最大元 (函数模板)

ranges::pop_heap (C++20)

移除最大堆中最大元 (算法函数对象)

make_heap

从元素范围创建最大堆 (函数模板)

ranges::make_heap (C++20)

从元素范围创建最大堆 (算法函数对象)

sort_heap

将最大堆变成按升序排序的元素范围 (函数模板)

ranges::sort_heap (C++20)

将最大堆变成按升序排序的元素范围 (算法函数对象)

is_heap

检查给定范围是否为最大堆 (函数模板)

ranges::is_heap (C++20)

检查给定范围是否为最大堆 (算法函数对象)

is_heap_until (C++11)

查找能成为最大堆的最大子范围 (函数模板)

ranges::is_heap_until (C++20)

查找能成为最大堆的最大子范围 (算法函数对象)

最小/最大操作

在标头 `<algorithm>` 定义

max

返回给定值中较大者 (函数模板)

ranges::max (C++20)

返回给定值中较大者 (算法函数对象)

max_element

返回范围中最大元 (函数模板)

ranges::max_element (C++20)

返回范围中最大元 (算法函数对象)

min

返回给定值中较小者 (函数模板)

ranges::min (C++20)
 返回给定值中较小者 (算法函数对象)

min_element
 返回范围中最小元 (函数模板)

ranges::min_element (C++20)
 返回范围中最小元 (算法函数对象)

minmax (C++11)
 返回两个元素间的较小者和较大者 (函数模板)

ranges::minmax (C++20)
 返回两个元素间的较小者和较大者 (算法函数对象)

minmax_element (C++11)
 返回范围中的最小元和最大元 (函数模板)

ranges::minmax_element (C++20)
 返回范围中的最小元和最大元 (算法函数对象)

clamp (C++17)
 在一对边界值下夹逼一个值 (函数模板)

ranges::clamp (C++20)
 在一对边界值下夹逼一个值 (算法函数对象)

字典序比较操作
 在标头 `<algorithm>` 定义

lexicographical_compare
 当一个范围字典序小于另一个时返回 `true` (函数模板)

ranges::lexicographical_compare (C++20)
 当一个范围字典序小于另一个时返回 `true` (算法函数对象)

lexicographical_compare_three_way (C++20)
 三路比较两个范围 (函数模板)

排列操作
 在标头 `<algorithm>` 定义

next_permutation
 生成元素范围的下一个字典序更大的排列 (函数模板)

ranges::next_permutation (C++20)
 生成元素范围的下一个字典序更大的排列 (算法函数对象)

prev_permutation
 生成元素范围的下一个字典序更小的排列 (函数模板)

ranges::prev_permutation (C++20)
 生成元素范围的下一个字典序更小的排列 (算法函数对象)

is_permutation (C++11)
 判断一个序列是否为另一个序列的排列 (函数模板)

ranges::is_permutation (C++20)
 判断一个序列是否为另一个序列的排列 (算法函数对象)

数值运算

在标头 `<numeric>` 定义

iota (C++11)

从初始值开始连续递增填充范围 (函数模板)

ranges::iota (C++23)

从初始值开始连续递增填充范围 (算法函数对象)

accumulate

求和或折叠范围中元素 (函数模板)

inner_product

计算两个范围中元素的内积 (函数模板)

adjacent_difference

计算范围中相邻元素的差 (函数模板)

partial_sum

计算范围中元素的部分和 (函数模板)

reduce (C++17)

类似 `std::accumulate`, 但不依序执行 (函数模板)

exclusive_scan (C++17)

类似 `std::partial_sum`, 第 *i* 个和中排除第 *i* 个输入 (函数模板)

inclusive_scan (C++17)

类似 `std::partial_sum`, 第 *i* 个和中包含第 *i* 个输入 (函数模板)

transform_reduce (C++17)

应用可调用对象, 然后乱序规约 (函数模板)

transform_exclusive_scan (C++17)

应用可调用对象, 然后计算排除扫描 (函数模板)

transform_inclusive_scan (C++17)

应用可调用对象, 然后计算包含扫描 (函数模板)